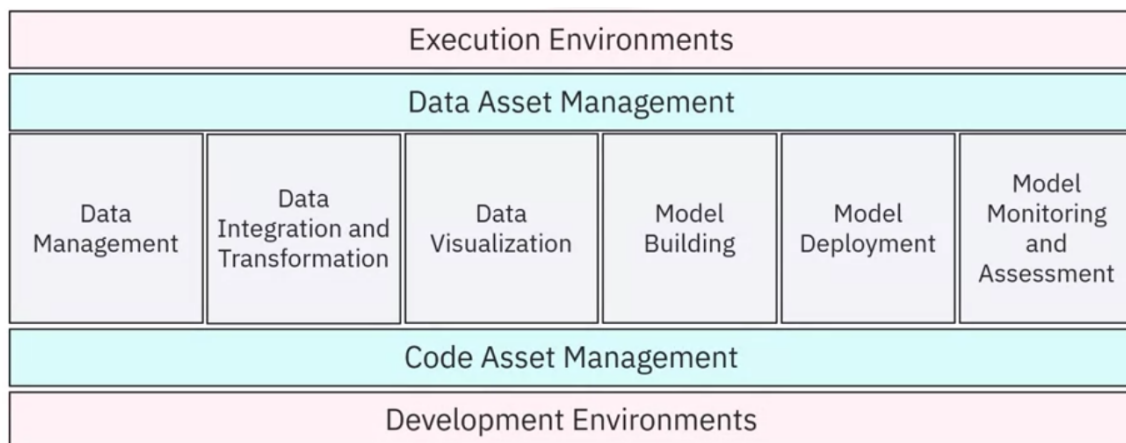


MODULE 1: Overview of Data Science Tools

In this module, you will learn about the different types and **categories of tools** that data scientists use and popular examples of each. You will also become familiar with Open Source, Cloud-based, and Commercial options for data science tools.

- Describe the components of a Data Scientist's toolkit and list various tool categories
- List examples of Open Source, Commercial, and Cloud-based tools in various categories

Categories of data Science tools.



Data Science **task categories**:

- data management,
- data integration and transformation,
- data visualization,
- model building,
- model deployment,
- model monitoring and assessment.

To do these tasks, you need **data asset management**, code asset management, execution environments, and development environments.

Let's see how each category enables you to make the best use of raw data.

Data management:

is the process of **collecting**, persisting, and **retrieving** data securely, efficiently, and cost-effectively. Data is collected from many sources, like Twitter, Flipkart, Media,

Sensors, and more. Store collected data in persistent storage so it is available whenever you need it.

Data Integration and Transformation

Is the process of Extracting, Transforming, and Loading data. This is called “ETL”. Some of this data is distributed in multiple repositories (database, cube, flat files). Use the Extraction process to extract data from these numerous repositories and save to a central repository like a Data Warehouse. Data Warehouses are primarily used to collect and store massive amounts of data for data analysis. Next, Data Transformation is the process of transforming the values, structure, and format of data. After extracting the data, the next step is to transform the data. In this example, height and weight data needs to be transformed to metric. And once the data is transformed, it’s time to load the data. Transformed data is loaded back to the Data Warehouse.

Data visualization

Conveys data more effectively for decision-makers.

Model building.

Train the data and analyze patterns with machine learning algorithms. You can then use this model to make predictions on new, unseen data.

Model building can be done using a service called **IBM Watson Machine Learning**.

Model deployment:

Integrating a developed model into a production environment, via APIs. helps make data-based decisions.

Model monitoring and assessment

Continuous quality checks to ensure a model’s accuracy, fairness, and robustness. (tools like Fiddler to track the performance of deployed models using metrics such as F1 score, true positive rate, or the sum of squared error).

A well-known example is the **IBM Watson Open scale**,

Code asset management

When you want to develop a model, you may need to update it, fix bugs, or improve code features incrementally. All of this **requires version control**. Developers use versioning to track and manage changes to a software project’s code.

When working on a model, **teams a centralized repository** where everyone can upload, edit, and manage the code files simultaneously. Collaboration allows diverse people to share and update the same project together.

GitHub, web-based, provides sharing, collaboration, and access control features.

Data asset management.

Also called **Digital AM**, is the organizing and managing of important data collected from different sources. Performed on a platform that allows versioning and

collaboration, replication, backup, and access right management for the stored data.

Development Environments,

Also called Integrated Development Environments, or

“IDEs”, provide a workspace and tools to **develop, implement, execute, test, and deploy** source code.

IDEs like **IBM Watson Studio** provide testing and simulation tools to emulate the real world so you can see how your code will behave after it is deployed.

Execution environment.

Has libraries to compile the source code and system resources that execute and verify the code. Cloud-based execution environments are not tied to any specific hardware or software, and offer tools like IBM Watson Studio for data preprocessing, model training, and deployment.

Finally, fully integrated visual tools like **IBM Watson Studio and IBM Cognos Dashboard Embedded cover all the previous tooling components**, and can be used to develop deep learning and machine learning models.

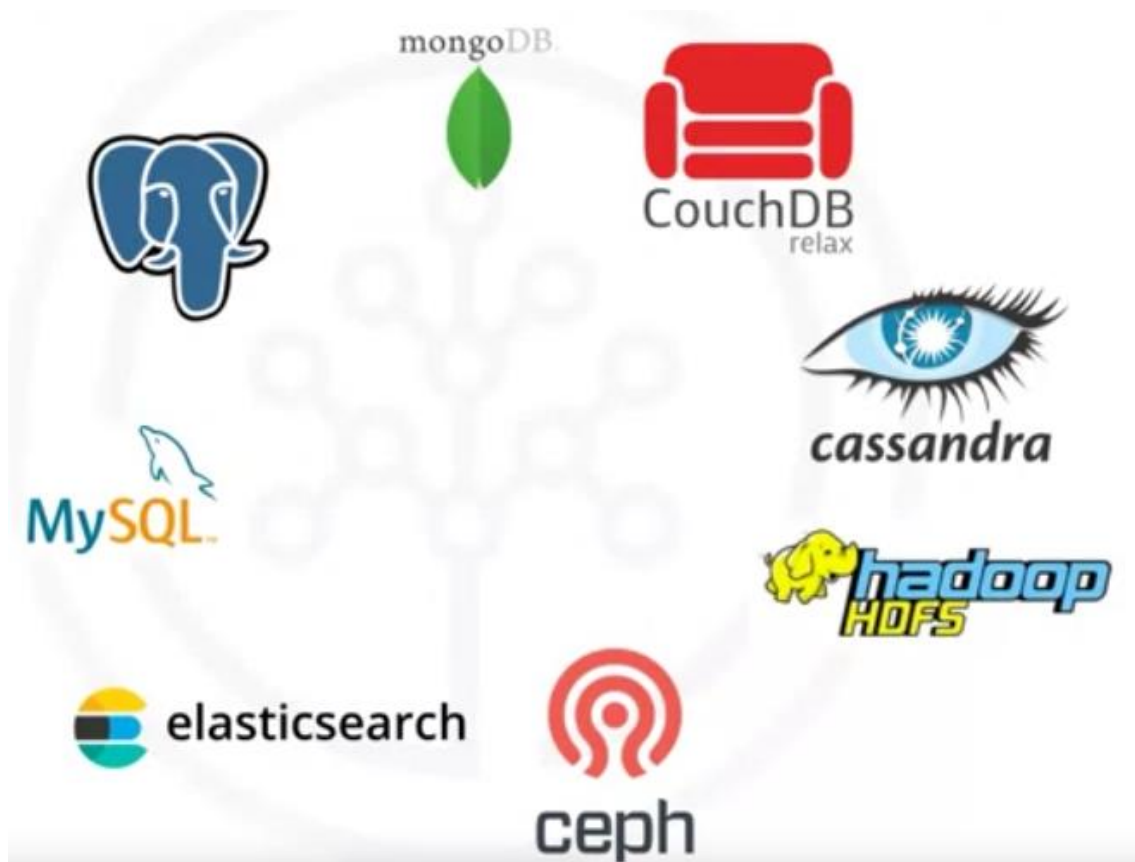
Summary

In this video, you learned that:

- The Data Science Task Categories are:
 - Data Management
 - Data Integration and Transformation
 - Data Visualization
 - Model Building
 - Model Deployment
 - Model Monitoring and Assessment
- Data Science Tasks are supported by:
 - Data Asset Management
 - Code Asset Management
 - Execution Environments
 - Development Environments

Open-source tools for data science

Data management



- **Relational** databases like MySQL (Popular web, warehousing e-commerce), and PostgreSQL (Powerful. Supports Json, spatial data, and **full text search**).
- **NoSQL** Databases like MongoDB (Document oriented, large volumens), Apache CouchDB (Document oriented uses JSON to store data, highly scalable, fault tolerant), and Apache Cassandra (Highly scalable, **distributed**, mission critical).
- **file-based** like the Hadoop File System (fault tolerant distributed storage and processing) or Cloud File systems like Ceph (scalable object, block and file storage).
- **Elastic search** tool (Highly scalable, distributed RESTful search an analutys engine based on lucene library) that stores text data, including the **real time creation of a search index** for fast document retrieval.

Data integration and transformation.

Extract, Transform, and Load (ETL) or Extract, Load, Transform (ELT). Another term for this process emerged: **Data Refinery and Cleansing**.



- Apache AirFlow, created by Airbnb originally.
- KubeFlow, Execution of data science pipelines on top of Kubernetes (distributed training and hyperparameter tuning).
- Apache Kafka, distributed streaming platform to, in real time, publish, process and subscribe to streams of records. Scalable, fault tolerant mission critical (originated from LinkedIn).
- Apache Nifi, which delivers a very nice web visual editor, automates flows of data between systems.
- Apache SparkSQL, uses ANSI SQL, dataframes and datasets and scales up to thousands of nodes (wide range of data sources, optimized performance for complex data processing tasks).
- NodeRED with a visual editor, runs on a Raspberry Pi. Event-driven flows of messages(devices, APIs, and web services)

Data visualization.



Distinguish between programming libraries or tools containing a user interface.

- Pixie Dust is also a python library but has a user interface that facilitates plotting in Python.
- Hue, no coding, can create visualizations from SQL queries and Hadoop.
- Kibana, a data exploration, and visualization web application is limited to Elasticsearch (data provider).
- Apache Superset is a enterprise ready data exploration and visualization Business intelligence web application (Geo spatial analysis and real time data processing).

Model building



Watson Studio
By IBM



IBM AutoAI



Watson OpenScale
By IBM



IBM Watson Machine Learning

-
- **IBM Watson Studio:** Engineered as an integrated environment, Watson Studio **simplifies developing, training, and deploying models**. It boasts support for multiple languages and frameworks, such as Python, R, and TensorFlow, alongside collaboration features, data preparation tools, and versatile deployment options.
- **IBM AutoAI:** A notable feature embedded within Watson Studio, IBM AutoAI streamlines the machine learning model construction process. By **dynamically exploring various algorithms and hyperparameters, it aims to identify the optimal model for a given dataset**.
- **IBM Watson OpenScale:** As a platform for overseeing and managing AI models in production, Watson OpenScale plays a pivotal role in ensuring **model fairness, explainability, and bias mitigation**. It furnishes insights into model performance and drift over time, facilitating informed decision-making.
- **IBM Watson Machine Learning:** Watson Machine Learning, available as a service on the IBM Cloud platform, enables users to **scale their training and deployment of machine learning models**. It seamlessly supports popular frameworks like TensorFlow, PyTorch, and scikit-learn, and offers APIs for seamless integration with other applications.

Model deployment.

Once you've created a machine learning model capable of predicting some critical aspects of the future, you should make it consumable by other developers and turn it into an API.



Apache PredictionIO (scalable and distributed infrastructure) currently only supports Apache Spark ML models (Recommendation, Classification, Clustering) for deployment, but support for all libraries is on the roadmap.

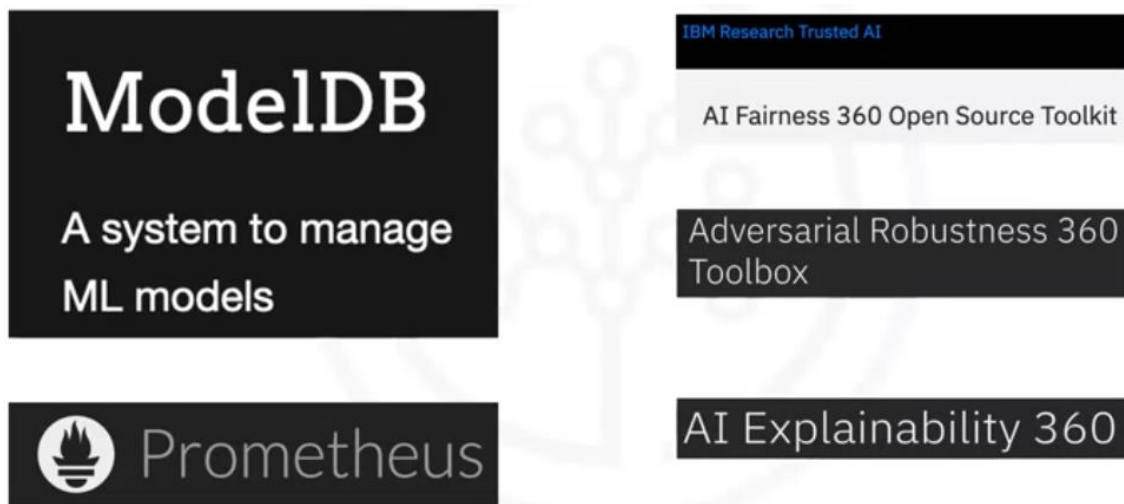
Seldon supports several frameworks (TensorFlow, Apache SparkML, R, and scikit learn) and runs on top of Kubernetes and Redhat OpenShift.

MLeap deploys SparkML models. **Export** models from different machine learning libraries.

TensorFlow service can serve any TensorFlow model. Either embedded in a Raspberry Pi or smartphone using **TensorFlow lite** or embedded in a web browser using **TensorFlow dot JS**.

Model monitoring and assessment.

Once you've deployed a machine learning model, you want to track its prediction performance while new data arrives to maintain outdated models.



Some examples are the following:

ModelDB is a machine model metadata base where information about the models is stored and queried. It natively supports Apache Spark ML Pipelines and scikit-learn. Tracks and reproduce experiments.

Prometheus not specifically made for machine learning model monitoring, it is used for this purpose. COLlects and stores metrics form different sources.

Model performance is measured by more than accuracy. **Model bias against protected groups like gender or race is important as well.**

The **IBM AI Fairness 360** open-source toolkit **detects and mitigates bias** in machine learning models.

Neural network-based deep learning models, can be subject to adversarial attacks (an attacker tries to mislead the model with manipulated data or by controlling it).

The **IBM Adversarial Robustness 360** Toolbox **detects vulnerability against adversarial attacks** and leverages the model to be more robust.

Finally, machine learning modes are often considered as a black box applying some magic.

The **IBM AI Explainability 360** toolkit addresses that problem by finding similar examples in a dataset to be presented to an end-user for manual comparison.

IBM AI Explainability 360 toolkit can also address the training of a simpler machine learning model to **explain the responsibility of different input variables directed toward the final decision of the model.**

Code Asset management



Git is now the de facto standard for code asset management, also known as version management or version control.

Around Git emerged several services. The most prominent is GitHub, but the runner-up is **GitLab**, with the advantage that the platform is entirely open source and **can be hosted and managed on your own**. Another choice is Bitbucket.

Data Asset Management

Also known as data governance or data lineage, is a crucial part of enterprise-grade data science. Data has to be versioned and annotated with metadata.



Apache Atlas is such a tool supporting this task for Hadoop ecosystems.

ODP Egeria, managed through the Linux Foundation, is an open ecosystem offers a set of open **APIs**, types, and interchange protocols **that metadata repositories use to share and exchange data**.

Kyro is an open-source data lake management software platform, with extensive support for data asset management tasks.

Summary

- Data management tools are MySQL, PostgreSQL, MongoDB, Apache CouchDB, Apache Cassandra, Hadoop File System, Ceph, and elastic search
- Data integration and transformation tools are Apache AirFlow, KubeFlow, Apache Kafka, Apache Nifi, Apache SparkSQL, and NodeRED
- Data Visualization tools are Pixie Dust, Hue, Kibana, and Apache Superset
- Model deployment tools are Apache PredictionIO, Seldon, Kubernetes, Redhat OpenShift, MLeap, TensorFlow service, TensorFlow lite, and TensorFlow dot JS
- Model monitoring tools are ModelDB, Prometheus, IBM AI Fairness 360, IBM Adversarial Robustness 360 Toolbox, and IBM AI Explainability 360
- Code asset management tools are Git, GitHub, GitLab, and Bitbucket
- Data asset management tools are Apache Atlas, ODPi Egeria, and Kylo

Development environments



Jupyter emerged as a tool for **interactive Python programming**, supports +100 different programming languages through “kernels.” (encapsulates the execution environment). **unifies documentation, code, output from the code, shell commands, and visualizations** in a single document.

Jupyter lab, next version of Jupyter Notebooks, will replace it. **Opens different types of files**, including Jupyter Notebooks, data, and terminals, and then arrange them **on the canvas**.

Apache Zeppelin, inspired by Jupyter Notebooks. and provides a similar experience. Integrates no-coding plotting capability without external plotting libraries.

RStudio is among the oldest development environments for statistics and data science (2011). It exclusively runs R and all associated R libraries. Python

development is possible. R is tightly **integrated into the Jupyter tool** and provides optimal user experience. RStudio **unifies programming, execution, debugging, remote data access, data exploration, and visualization into one tool.**

Spyder mimic RStudio for Python. **Jupyter is used more.** Integrates **code, documentation, and visualizations, among others, into a single canvas.**

Sometimes your data doesn't fit into a single computer's storage or main memory capacity.

Cluster execution environments.



Apache Spark key property is **linear scalability**. It is a **batch** engine processing vast amounts of data one by one or file by file

Apache Flink was developed after Apache Spark is a **stream**-processing engine with its main focus on processing real-time data streams.

Apache Spark is the choice for most use cases.

Ray focus on large-scale deep learning model training.

Fully integrated and visual environments.



No programming knowledge is necessary. The tools support a subset of important tasks that include **data integration and transformation, data visualization, and model building**.

KNIME (University of Konstanz in 2004) **visual drag-and-drop capabilities**. It has built-in visualization capabilities. In addition, it can be extended by programming in R and Python and even has connectors to Apache Spark.

Orange is less flexible than KNIME but is easier to use.

Commercial tools for data Science

Data management



ORACLE[®]
DATABASE



Microsoft[®]
SQL Server[®]



IBM
DB2

Since data is the heart of every organization, **commercial support availability** plays a major role. (software vendors, influential partners, and support networks).

Data integration tools.



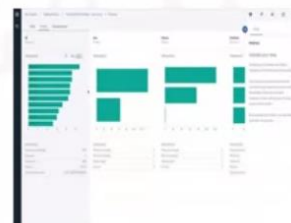
Informatica[™]



talend



IBM
InfoSphere DataStage



IBM
Watson Studio Desktop

Informatica PowerCenter and IBM InfoSphere DataStage are the leaders, followed by SAP, Oracle, SAS, Talend, and Microsoft products.

These tools support the design and deployment of **ETL data processing pipelines through a graphical interface**. They bring along **connectors** to most of the commercial and open-source target information systems.

Finally, Watson Studio Desktop includes a component called **Data Refinery**, which enables definition and execution of data integration processes in a spreadsheet-style.

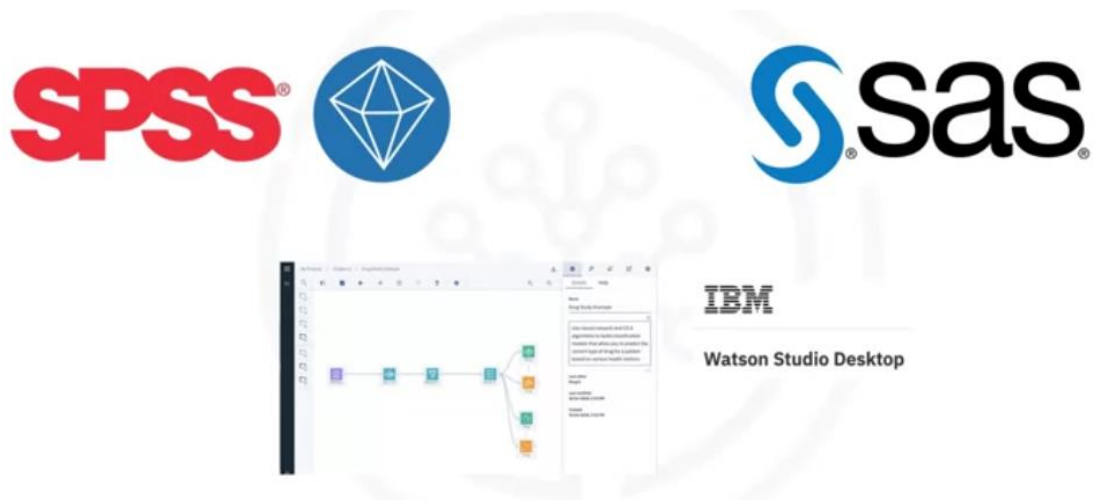
Data visualizations



For end users: Tableau, Microsoft Power BI, and IBM Cognos Analytics.

For data scientists, the visualization can show relationships between different columns in a table. This functionality is contained in Watson Studio Desktop.

Model BUILDing

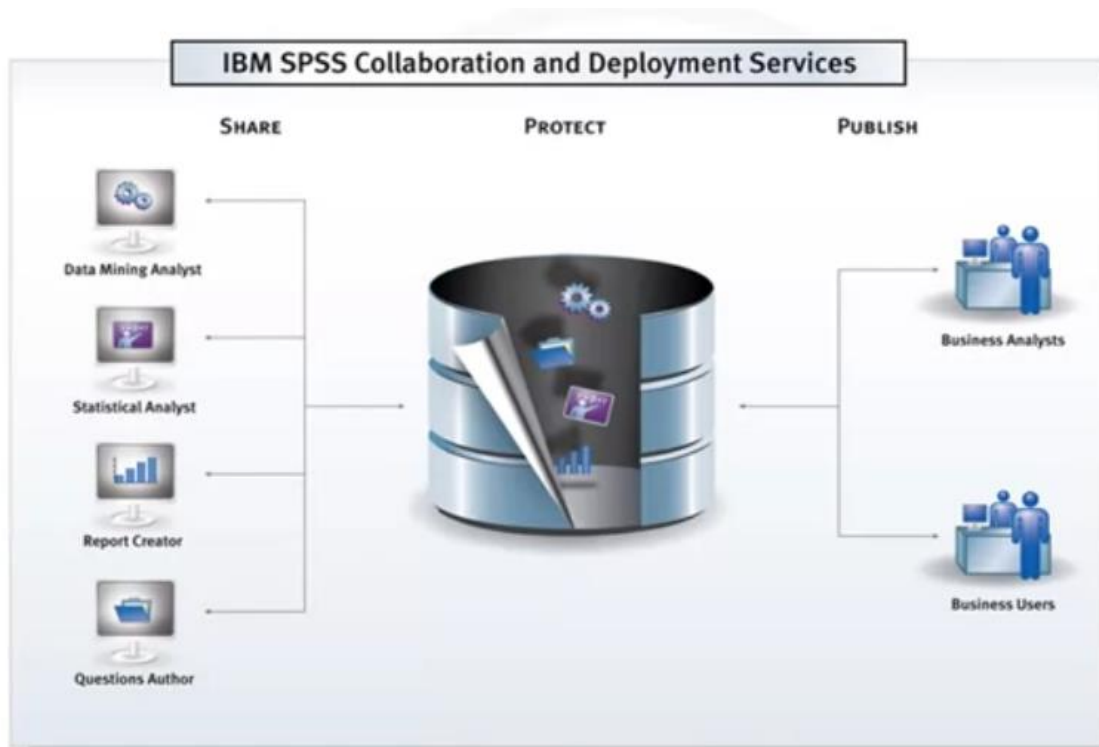


Use a data mining product.

SPSS Modeler is also available in Watson Studio Desktop,

Model deployment

in commercial software is **tightly integrated into the model-building** process.



Deploy any type of asset created by the SPSS software tools suite.

The same holds for other vendors.

Model monitoring

New discipline. Without relevant commercial tools, **open source is the first choice.**

Code Asset management

The same is true for code asset management. Open source with Git and GitHub is the de facto standard.

Data asset management

Data governance or data lineage,

is a crucial part of enterprise-grade data science.



Informatica™



Data versioned and annotated

with metadata.

Data dictionary

facilitates the discovery of data assets.

Each data asset is assigned to a **data steward or the data owner**. The data owner is responsible for that data asset and can be contacted.

Data lineage

is covered, **allowing tracking back the transformation steps** in creating the data assets. The data lineage also includes a **reference to the actual source** data.

Data privacy and retention

Rules and policies can be added to reflect complex regulatory and business requirements

Development environments

Watson Studio is a fully integrated development environment for data scientists. Most people consume it through the **cloud**. And there is also a **desktop version available** that combines Jupyter Notebooks with graphical tools.

Fully integrated visual tools



Watson Studio, together with Watson Open Scale, is a fully integrated tool covering the data science life cycle involving all tasks discussed previously. They can be deployed in a local data center, on top of Kubernetes / RedHat OpenShift.

H2O Driverless AI, which covers the complete data science life cycle.

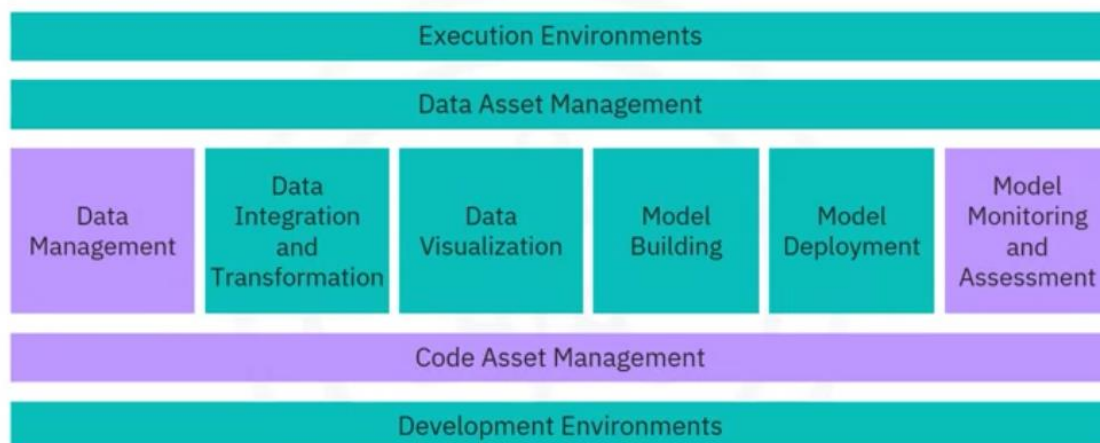
Summary

In this video, you learned that:

- Commercial tools support the most common tasks in data science
- Data management tools are Oracle Database, Microsoft SQL Server, and IBM Db2
- Data integration tools are mainly provided by Informatica PowerCenter, IBM InfoSphere DataStage. These are followed by products from SAP, Oracle, SAS, Talend, Microsoft, and Watson Studio Desktop
- Model building tools are SPSS Modeler, and SAS enterprise miner
- SPSS Modeler is also available in Watson Studio Desktop
- Data asset management tools are provided by Informatica and IBM
- Watson Studio, together with Watson Open Scale is a fully integrated tool covering the data science life cycle

Cloud base tools for Data Science

Since cloud products are a newer species, they follow the trend of having multiple tasks integrated in tools. This integration is applicable for the tasks marked green in the diagram.



Fully integrated visual tools and platforms category.

Since these tools introduce a component where large-scale execution of data science workflows happens in **compute clusters**, we have changed the title and added the word **“Platform.”**

These clusters are composed of **multiple server machines, transparently for the user, in the background.**

Watson Studio and Watson OpenScale cover the complete development life cycle for all data science, machine learning, and artificial intelligence (AI) tasks.

Microsoft Azure Machine Learning. It is also a full cloud-hosted offering supporting the complete development life cycle of all data science, machine learning, and AI tasks.

H2O Driverless AI. Although it is a product you download and install, there exists a one-click deployment for the standard cloud service providers. Since the **cloud**

provider does not do operations and maintenance, as with Watson Studio, Open Scale, and Azure Machine Learning, this delivery model should be distinct from Platform or Software as a service - PaaS or SaaS.

Data management

With some exceptions, software-as-a-service (SaaS) versions of existing open source and commercial tools exist.

The cloud provider operates the tool for you in the cloud. **backing up your data and configuring and installing updates.**

Some **proprietary tools are only available from a single cloud provider.**

One example of such a service is **Amazon Web Services DynamoDB**, which is a NoSQL database. It allows storage and retrieving data in a **key-value** or a document store format. The most prominent document data structure is **JSON**.

Another flavor of such a service is **Cloudant**, which is a **database as a service** offering. But, in the background, it is based on the open-source Apache CouchDB. The advantage is that complex operational tasks like **updating, backup, restoring, and scaling are done by the cloud provider.**

However, the Cloudant service offering is compatible with CouchDB. Therefore, the application migrates to another CouchDB server without making any changes to the application.

IBM offers Db2 as a service as well. It is an example of a commercial database made available as a SaaS offering in the cloud, **taking away operational tasks from the user.**

Data integration tools



Informatica

IBM

Data Refinery

It means the **transformation steps are not done by a data integration team but are pushed toward the domain of the data scientist or data engineer.**

Data Refinery is part of IBM Watson Studio. it allows transforming large amounts of raw data into consumable, quality information in a spreadsheet-like user interface.

Data visualization

Datameer

IBM Cognos Analytics



IBM Watson Studio

An example of a smaller company offering a cloud-based data visualization tool Datameer.

IBM offers its famous Cognos Business intelligence suite as a cloud solution.

IBM Data Refinery also offers data exploration and visualization functionality in Watson Studio.

hierarchical edge bundling that depicts correlations and affiliations between entities.

If sufficient, a classic bar chart, 2D scatter plot tree map, pie chart, word cloud

Model building

Every cloud provider has a solution for this task.

Watson Machine Learning can train and build models using various open-source libraries.

Google has a similar service on their cloud called AI Platform Training.

Model deployment

in commercial software is usually tightly integrated into the model-building process.

Here is an example of the SPSS Collaboration and Deployment Services which can be used to deploy any asset created by the SPSS software tools suite.

The same holds for other vendors.

In addition, commercial software can export models in an open format.

For example, **SPSS Modeler** supports exporting models as **Predictive Model Markup Language (PMML)**, which other commercial and open software packages can read.

In addition, **Watson Machine Learning** deploys a model and makes it available to consumers using a REST interface.

Model monitoring and assessment



Amazon SageMaker Model Monitor

Watson OpenScale

Amazon SageMaker Model Monitor is an example of a cloud tool to monitor deployed machine learning and deep learning models continuously.

Another tool for model monitoring is **Watson OpenScale**.

Everything marked in green can be done using Watson Studio and OpenScale.

Summary

In this video, you've learned:

- Watson Studio and Watson OpenScale cover the complete development life cycle for all data science, machine learning, and AI tasks
- In data management, with some exceptions, there exists a software-as-a-service (SaaS) version of existing open source and commercial tools
- Two commercial data integration tools widely used are Informatica Cloud Data Integration and IBM's Data Refinery
- An example of a cloud-based data visualization tool is Datameer and IBM's Congos Business intelligence suite
- Model building can be done using a service such as Watson Machine Learning
- Amazon SageMaker Model Monitor is an example of a cloud tool to monitor deployed machine learning and deep learning models continuously

Module Summary

Congratulations! You have completed this module. At this point in the course, you know:

- The **Data Science Task Categories** include:
 - Data Management - storage, management and retrieval of data
 - Data Integration and Transformation - streamline data pipelines and automate data processing tasks
 - Data Visualization - provide graphical representation of data and assist with communicating insights
 - Modelling - enable Building, Deployment, Monitoring and Assessment of Data and Machine Learning models
- Data Science Tasks support the following:
 - Code Asset Management - store & manage code, track changes and allow collaborative development
 - Data Asset Management - organize and manage data, provide access control, and backup assets
 - Development Environments - develop, test and deploy code
 - Execution Environments - provide computational resources and run the code

The data science ecosystem consists of many open source and commercial options, and include both traditional desktop applications and server-based tools,

as well as cloud-based services that can be accessed using web-browsers and mobile interfaces.

Data Management Tools: include Relational Databases, NoSQL Databases, and Big Data platforms:

- MySQL, and PostgreSQL are examples of Open Source Relational Database Management Systems (RDBMS), and IBM Db2 and SQL Server are examples of commercial RDBMSes and are also available as Cloud services.
- MongoDB and Apache Cassandra are examples of NoSQL databases.
- Apache Hadoop and Apache Spark are used for Big Data analytics.

Data Integration and Transformation Tools: include Apache Airflow and Apache Kafka.

Data Visualization Tools: include commercial offerings such as Cognos Analytics, Tableau and PowerBI and can be used for building dynamic and interactive dashboards.

Code Asset Management Tools: Git is an essential code asset management tool. GitHub is a popular web-based platform for storing and managing source code. Its features make it an ideal tool for collaborative software development, including version control, issue tracking, and project management.

Development Environments: Popular development environments for Data Science include Jupyter Notebooks and RStudio.

- Jupyter Notebooks provides an interactive environment for creating and sharing code, descriptive text, data visualizations, and other computational artifacts in a web-browser based interface.
- RStudio is an integrated development environment (IDE) designed specifically for working with the R programming language, which is a popular tool for statistical computing and data analysis.

MODULE 2: Languages of data science

This module will bring awareness about the criteria determining which language you should learn. You will learn the **benefits of** Python, R, SQL, and other common languages such as Java, Scala, C++, JavaScript, and Julia. You will explore how you can use these languages in Data Science. You will also look at some **sites for more information about the languages**.

Learning Objectives

Identify the criteria and roles for determining the language to learn.

Identify the users and benefits of Python.

Identify the users and uses of the R language.

Define SQL elements and list their benefits.

Review languages such as Java, Scala, C++, JavaScript, and Julia.

List the global communities for connecting with other users.

Language of data Science.

Criteria for determining the language to learn, and list the roles involved in data science.

For users who are just starting on their data science journey, the range of available technical options can be overwhelming. There are literally thousands of different programming languages with their own strengths and weaknesses.



Which language should you learn first?

The answer to that question depends largely on your needs, the problems you are trying to solve, and who you are solving them for.

First and foremost, we recommend considering the Python, R, and SQL languages.

However, other languages such as Scala, Java, C++, and Julia with specific features are also popular.

JavaScript, PHP, Go, Ruby, and Visual Basic all have their own unique use cases as well.

The language you choose to learn will **depend on the things you need to accomplish and the problems you need to solve.**

The problems can be related to the company you work for, what role you have, and the age of your existing application. So, what problems do you need to solve?

We'll have a better insight into the answers to this question as we learn

more about the popular languages in the data science industry.

roles in data science,

- Business Analyst,
- Database Engineer,
- Data Analyst,
- Data Engineer,
- Data Scientist,
- Research Scientist,
- Software Engineer,
- Statistician,
- Product Manager,
- Project Manager, and so on.
-

In this video, you learned that you should select a language to learn depending on your needs, the problems you are trying to solve, and who you are solving them for.

The popular languages are Python, R, SQL, Scala, Java, C++, and Julia.

JavaScript, PHP, Go, Ruby, and Visual Basic all have their own unique use cases as well.

The problems you need to solve can be related to your company, role, and **age of the existing application**.

Introduction to Python.

Python is a powerhouse of a language. An aspiring data scientist should learn it first. It is the most widely used and most popular programming language used in the data science industry.

Who is Python for

You **already know how to program**, it uses clear and readable syntax. You can develop the same programs from other languages **with lesser code using Python**.

For **beginners**, Python is a good language to **start with because of the huge** global community and wealth of documentation.

80% of data professionals use Python worldwide.

Python is **useful in many areas** including data science, AI and machine learning, web development, and Internet of Things (IoT) devices, like the Raspberry Pi.

Large organizations heavily use python.

Python is **widely supported by a global community** and shepherded by the Python Software Foundation.

What make Python great.

Python is a **high-level, general-purpose programming language** that can be applied to many different classes of problems.

It has a large, **standard library that provides tools suited to many different tasks** including but not limited to Databases, Automation, Web scraping, Text processing, Image processing, Machine learning, and Data analytics.

For **data science**, you can use Python's scientific computing libraries like Pandas, NumPy, SciPy, and Matplotlib.

For artificial intelligence, it has TensorFlow, PyTorch, Keras, and Scikit-learn.

Python can also be used for **Natural Language Processing (NLP)** using the Natural Language Toolkit (NLTK).

Diversity and inclusion efforts

The Python language has a **code of conduct executed by the Python Software Foundation** that seeks to ensure safety and inclusion for all, in both online and in-person Python communities.

Communities like PyLadies seek to create spaces for people interested in learning Python in safe and inclusive environments.

PyLadies is an international mentorship group with a focus on helping more women become active participants and leaders in the Python open-source community.

Summary

In this video, you learned that:

- Python uses clear and readable syntax
- Python has a huge global community and a wealth of documentation
- For data science you can use Python's scientific computing libraries like Pandas, NumPy, SciPy, and Matplotlib
- Python can also be used for Natural Language Processing (NLP) using the Natural Language Toolkit (NLTK)
- Python community has a well-documented history of paving the way for diversity and inclusion efforts in the tech industry as a whole

Introduction to R.

Learning three languages can earn you an increment!

Open Source vs. Free Software

Now, Python is open source, while R is free software.

Both, open source and free software, are **free to use**, refer to the **same** set of **licenses** (GNU).support **collaboration**.

And in many cases, these terms can be used interchangeably (but not all).

The Open-Source Initiative (OSI) champions open source, while the Free Software Foundation (FSF) defines free software. **Open source** is more **business focused**, while **free software** is more **values focused**.

Why R?

Because it is **free software**.

Allows for **private use, commercial use, and public collaboration**.

Supported by a wide global community.

Who is R for?

Statisticians, mathematicians, and data miners who develop statistical software, graphing, and data analysis, **Learn R after a few years** into their data science career.

R Language's array-oriented syntax makes it **easier to translate from math to code** for learners with no or minimal programming background.

Popular in academia.

Large companies use it.

What makes R great?

R has become the **world's largest repository of statistical knowledge**. R had more than 15,000 publicly released packages making it possible to conduct complex exploratory data analysis.

R **integrates well with other computer languages** like C++, Java, C, .Net, and Python.

Using R, common mathematical operations like matrix multiplication give immediate results.

And R has **stronger object-oriented programming facilities** than most statistical computing languages.

Global Communities

Now, there are many ways to connect with other R users around the globe.

For connecting to other users, you can use communities such as useR, WhyR, SatRdays, and R-ladies.

And in addition, you can check out the R project website for R conferences and events.

Summary

In this video, you learned that:

- The Open Source Initiative (OSI) champions open source, while the Free Software Foundation (FSF) defines free software
- Python is open source, and R is free software
- R language's array-oriented syntax makes it easier to translate from math to code for learners with no or minimal programming background
- R has become the world's largest repository of statistical knowledge

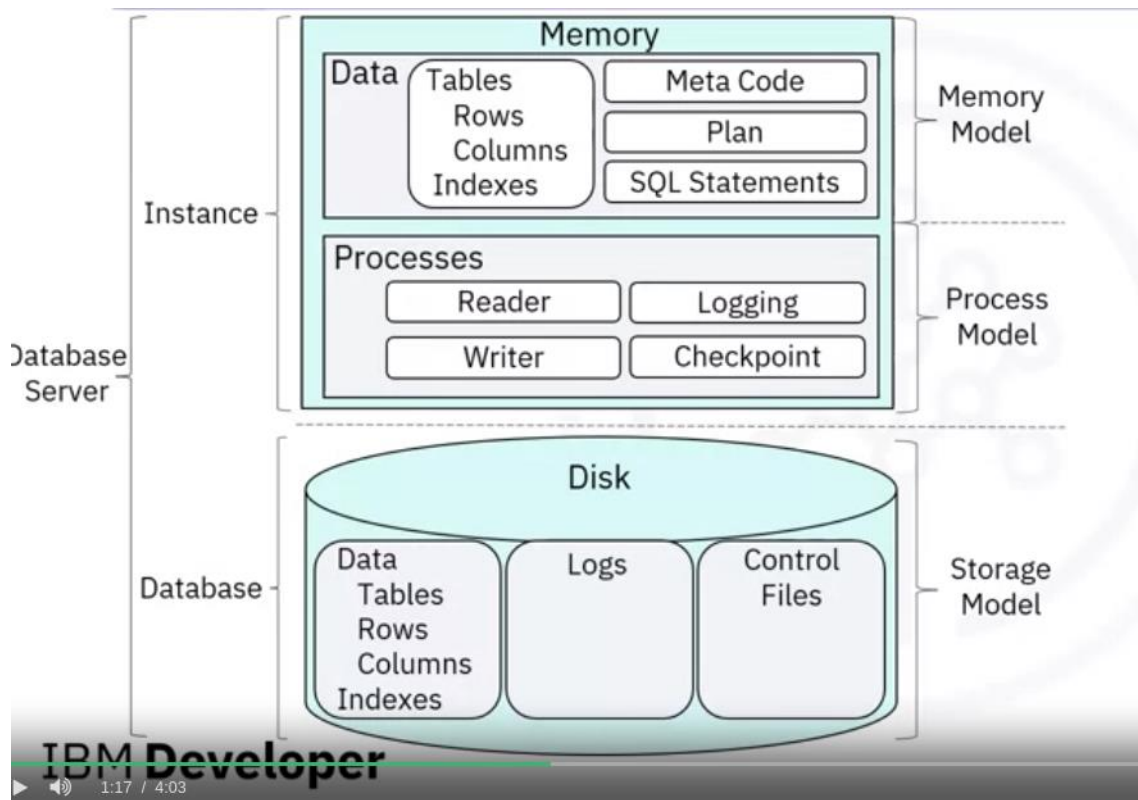
Introduction to SQL.

What is SQL?

Appeared in 1974 and was developed at IBM. SQL is a bit different than the other languages. it is a **non-procedural language**. Its scope is **limited to querying and managing structured data in relational databases**.

SQL interfaces have **also been developed for many NoSQL** and big data repositories.

Language elements, including: Clauses, Expressions, Predicates, Queries, and Statements.



What makes SQL Great

Knowing SQL will help you **get many different jobs** in data science, such as a business and data analyst. This knowledge is also a must in data engineering.

When performing operations with SQL, the data is accessed directly, **without needing to copy** the data separately, which can considerably **speed up workflow executions**.

SQL behaves like an **interpreter between you and the database**.

SQL is an American National Standards Institute (or **ANSI**) standard,

Many SQL Databases Available



MySQL, IBM DB2, PostgreSQL, **Apache Open Office Base**, SQLite, Oracle, MariaDB, Microsoft SQL Server, and more.

The **syntax** of the SQL you write **may change** based on the relational database management system you are using.

If you want to learn SQL, you should focus on a specific relational database and then plug into the community for that specific platform.

In addition, there are many available great introductory courses on SQL!

Summary

In this video, you learned that:

- SQL is different from other software development languages because it is a non-procedural language
- SQL's scope is limited to querying and managing data
- SQL was designed for managing data in relational databases
- SQL behaves like an interpreter between you and the database
- If you learn SQL and use it with one database, you can apply your SQL knowledge with many other databases easily

Other languages for Data Science.

Other languages



Java

General-purpose object-oriented programming language. designed to be fast and scalable, compiled to bytecode and **run on the Java Virtual Machine** or JVM.

Data science tools built with Java include:

Weka for data mining, **Java-ML** for machine learning, **Apache MLlib** makes machine learning scalable, and **Deeplearning4j** for deep learning.

Hadoop is another application of Java which manages data processing and storage for big data applications running in clustered systems.

Scala

A combination of **scalable and language**, it is **General-purpose programming language that provides support for functional programming and is a strong static type system**, runs on the JVM.

Program built with Scala is **Apache Spark**. which make parallel jobs easy to write.

It has an optimized engine that supports general computation graphs.

Spark includes **Shark**, which is a query engine, **MLlib** for machine learning, **GraphX** for graph processing, and **Spark Streaming**. It was designed to be faster than Hadoop.

C++

A **general-purpose programming language**. improves processing speed, enables system programming, and provides broader control over the software application.

Many organizations that use Python or other high-level languages for data analysis and exploratory tasks rely on C++ to develop programs that feed data to customers in real-time.

For data science, **TensorFlow** is a popular Deep Learning library for dataflow that was built with C++. Although C++ is the foundation of TensorFlow, it runs on a python interface, so users don't require the knowledge of C++ to run it.

MongoDB is a NoSQL database for big data management that was built with C++.

Caffe is a deep learning algorithm repository built with C++ with Python and Matlab bindings.

JavaScript

A **general-purpose language** that extended beyond the browser with the creation of **Node.js** and other server-side approaches.

Javascript is NOT related to the Java language.

For Data Science, undoubtedly **TensorFlow.js** makes machine learning and deep learning possible in Node.js as well as in the browser.

TensorFlow.js was also adopted by other open-source libraries including brain.js and machinelearn.js.

Another implementation of JavaScript for Data Science is **R-js**. The project R-js has **re-written linear algebra specifications from the R Language into typescript**.

This sets the foundation for future projects to implement more powerful math base frameworks like Numpy and SciPy of Python.

Typescript is a superset of JavaScript.

Julia

Designed at MIT for high-performance **numerical analysis and computational science**, **provides speedy development** like Python or R, while **producing programs that run as fast as C** or Fortran programs.

It's **compiled** which means that Julia code is executed directly on the processor as executable code.

It calls C, Go, Java, MATLAB, R, Fortran, and Python libraries, and has refined parallelism.

Julia as a language is only 8 years old, written in 2012, but there is a lot of promise for its future impact on the data science industry.

One great application of Julia for Data Science is **JuliaDB**, which is a package for working with large persistent data sets.

Summary

Built with **Java** : Weka, Java-ML, Apache MLlib, and Deeplearning4.

Built with Scala: Apache Spark, that includes Shark, MLlib, GraphX, and Spark Streaming.

Built with **C++** : TensorFlow, MongoDB and Caffe.

Built with **JavaScript**: TensorFlow.js and R-js.

Built with **Julia**: JuliaDB.

MODULE 3: Packages, APIs, DataSets and Models

This module will give you **in-depth knowledge of different libraries, APIs, dataset sources and models** used by data scientist.

Learning Objectives

- List examples of the various libraries: scientific, visualization, machine learning, and deep learning.
- Define REST API to request and respond.
- Describe data sets and sources of data.
- Explore open data sets on the Data Asset eXchange.
- Describe how to use a learning model to solve a problem.
- List the tasks that a data scientist needs to perform to build a model.

- Explore ML models in the Model Learning eXchange.

Libraries for Data Science.

Libraries are a collection of **functions and methods** that allow you to perform many **actions without writing the code**.

Scientific Computing Libraries in Python

Also called frameworks, libraries contain built-in modules providing different functionalities, which you can use directly.

Pandas

Offers data structures and tools for effective **data cleaning, manipulation, and analysis different types of data**. The primary instrument, a two-dimensional table consisting of columns and rows, called a **Data Frame**. Pandas can also provide **easy indexing** so you can work with your data.

NumPy

libraries are based on arrays and matrices, allowing you to apply mathematical functions to the arrays. **Pandas is built on top of NumPy**.

Visualization Libraries in Python.

To communicate with others and display meaningful results of an analysis.

Matplotlib

The most well-known library for data visualization. The graphs are easily customizable.

Seaborn

Based on matplotlib generates heat maps, time series, and violin plots.

Machine Learning and Deep Learning Libraries in Python.

Scikit-learn

Contains tools for statistical modeling, including regression, classification, clustering, and so on. It is built on NumPy, SciPy, and matplotlib. It is **simple to get started**. In this high-level approach, you **define the model and specify the parameter** types you want to use.

Keras

Allows you to build the **standard deep learning model** in a quick, simple manner.

It can function using Graphics processing units (GPU) but in many cases, a lower-level environment is necessary for deep learning.

Deep Learning Libraries in Python

TensorFlow

A low-level framework used in the large-scale production of deep learning models. It's designed for **production and deployment**.

Pytorch

For **experimentation**, making it simple for researchers to test ideas.

Scala Libraries

Scala is predominately used in **data engineering and data science**.

Apache Spark

A general-purpose cluster-computing framework allowing you to process in parallel in more than one computer simultaneously. The Spark library **has similar functionality to** Pandas, Numpy, and Scikit-learn. Apache Spark data processing **jobs can be in: Python R Scala, and SQL**

Vegas

For **statistical data visualizations** of data files and Spark Data Frames.

Big DL.

For deep learning, you can use

R libraries

R has built-in functionality for machine learning and data visualization,

Ggplot2 is a popular library for data visualization in R.

Libraries that allow you to interface with Keras and TensorFlow.

And **R was a de-facto standard for open-source data science, but now Python will supersede it.**

Summary

- Libraries usually contain built-in modules providing different functionalities that can be used directly
- You can use data visualization methods to communicate with others and display meaningful results of an analysis
- For machine learning, the Scikit-learn library contains tools for statistical modeling including regression, classification, clustering and so on
- TensorFlow is a low-level framework used in large scale production of deep learning models
- Apache Spark is a general-purpose cluster-computing framework allowing you to process data using compute clusters

Application Programming Interfaces.

An application programming interface (API) allows communication between two pieces of software without knowing what happens at the backend.

The API only refers to the interface. It is the part of the library you see while it contains all the program components. The software component at the back end can be the same, but there can be **an API for different languages**.

Consider TensorFlow at the backend, written in C++ that can use APIs for other languages, such as: Python, JavaScript, C++, Java, Go, Julia, Matlab, R, Scala

REST APIs

The RE stands for **Representational**. The S stands for **State**. The T stands for **Transfer**.

They allow you to **communicate through the internet** and take advantage of resources like storage, data, artificially intelligent algorithms, and much more.

In Rest API, **your program is the client**. The API communicates with a web service.

Though **there are rules regarding Communication**, Input or Request, and Output or Response.

The client finds the service via an endpoint.

Data is transmitted over the internet **using HTTP methods**.

The request is sent using an **HTTP message that contains a JSON file**.

The file contains instructions for what operation is to be performed by the web service.

This operation is transmitted to the web service via the internet.

And the service performs the operation.

Similarly, the web service returns a response through an HTTP message,

where the information is returned using a JSON file.

And this information is transmitted back to the client.

Watson Text to Speech API.

This API converts speech to text. A POST request sends a copy of the audio file. The API will send the text transcription.

Watson language-translator API.

Send text to translate. The API will translate the text and send the translation back to you.

Summary

- An application programming interface (API) allows communication between two pieces of software
- API is the part of the library you see while the library contains all the components of the program
- REST APIs allow you to communicate through the internet and take advantage of resources like storage, data, artificially intelligent algorithms, and much more

Data Sets – Powering Data Science.

What is a data set?

A data set is a **structured collection of data** (text, numbers, images, audio, or video files).

A **tabular data set** comprises a collection of rows containing columns that store the information (one observation). CSV

One popular tabular data format is "comma separated values," or CSV.

Hierarchical or network data structures represent relationships between data. is organized in a tree-like format, whereas network data is stored as a graph.

A data set might also include **raw data files**, such as images or audio.

The Modified National Institute of Standards and Technology (MNIST) dataset is commonly used to train image processing systems with images of handwritten digits.

Data Ownership

Private

Traditionally, most data sets were private because they contained **proprietary** or **confidential** information such as customer data, pricing data, or other **commercially sensitive** information.

Open Data

Open data has played a significant role in the growth of data science, machine learning, and artificial intelligence providing free information to uncover previously unknown and potentially valuable insights.

It has allowed practitioners to hone their skills in various data sets.

Where to find open data

Open data portals <http://dataportals.org/>

United Nations <https://data.un.org/>

USA <https://data.gov/>

EU <https://data.europa.eu/en>

Kaggle <https://www.kaggle.com/datasets>

Google <https://datasetsearch.research.google.com/>

These are called open datasets.

Open datasets include a public license and are very useful for your journey as a Data Scientist. Some of the most informative open dataset sources are listed below.

Government Data:

<https://www.data.gov/>

<https://www.census.gov/data.html>

<https://data.gov.uk/>

<https://www.opendatane트워크.com/>

<https://data.un.org/>

Financial Data Sources:

<https://data.worldbank.org/>

<https://www.globalfinancialdata.com/>

<https://comtrade.un.org/>

<https://www.nber.org/>

<https://fred.stlouisfed.org/>

Crime Data:

<https://www.fbi.gov/services/cjis/ucr>

<https://www.icpsr.umich.edu/icpsrweb/content/NACJD/index.html>

<https://www.drugabuse.gov/related-topics/trends-statistics>

<https://www.unodc.org/unodc/en/data-and-analysis/>

Health Data:

<https://www.who.int/gho/database/en/>

<https://www.fda.gov/Food/default.htm>

<https://seer.cancer.gov/faststats/selections.php?series=cancer>

<https://www.opensciencedatacloud.org/>

<https://pds.nasa.gov/>

<https://earthdata.nasa.gov/>

<https://www.sгим.org/communities/research/dataset-compendium/public-datasets-topic-grid>

Academic and Business Data:

<https://scholar.google.com/>

<https://nces.ed.gov/>

<https://www.glassdoor.com/research/>

<https://www.yelp.com/dataset>

Other General Data:

<https://www.kaggle.com/datasets>

<https://www.reddit.com/r/datasets/>

Propriety datasets and sources

Proprietary datasets contain data primarily owned and controlled by specific individuals or organizations. This data is limited in distribution because it is sold with a licensing agreement.

Some data from private sources cannot be easily disclosed, like public data.

National security data, geological, geophysical, and biological data are examples of propriety data. Copyright laws or patents usually bind this type of data. Proprietary datasets that mainly contain sensitive information are less widely available than open datasets.

Some standard propriety dataset sources are listed below.

Health Care:

<https://www.sгим.org/communities/research/dataset-compendium/proprietary-datasets>

Financial Market data:

<https://datarade.ai/data-categories/proprietary-market-data>

Google Cloud based datasets:

<https://cloud.google.com/datasets>

Dataset licenses

When you select a dataset, it is necessary to look into the license. A license explains whether you can use that dataset or not; or explains if you have to accept certain guidelines to use that dataset. The different license types are listed below.

PUBLIC DOMAIN MARK - PUBLIC DOMAIN

When a dataset has a Public Domain license, all the rights to use, access, modify and share the dataset are open to everyone. Here there is technically no license.

OPEN DATA COMMONS PUBLIC DOMAIN DEDICATION AND LICENSE – PDDL

Open Data Commons license has the same features as the Public Domain license, but the difference is the PDDL license uses a licensing mechanism to give the rights to the dataset.

CREATIVE COMMONS ATTRIBUTION 4.0 INTERNATIONAL CC-BY

This license allows users to share and modify a dataset, but only if they give credit to the creator(s) of the dataset.

COMMUNITY DATA LICENSE AGREEMENT – CDLA PERMISSIVE-2.0

Like most open-source licenses, this license allows users to use, modify, adapt, and share the dataset, but only if a disclaimer of warranties and liability is also included.

OPEN DATA COMMONS ATTRIBUTION LICENSE - ODC-BY

This license allows users to share and adapt a dataset, but only if they give credit to the creator(s) of the dataset.

CREATIVE COMMONS ATTRIBUTION-SHAREALIKE 4.0 INTERNATIONAL - CC-BY-SA

This license allows users to use, share, and adapt a dataset, but only if they give credit to the dataset and show any changes or transformations, they made to the dataset. Users might not want to use this license because they have to share the work they did on the dataset.

COMMUNITY DATA LICENSE AGREEMENT – CDLA-SHARING-1.0

This license uses the principle of ‘copyleft’: users can use, modify, and adapt a dataset, but only if they don’t add license restrictions on the new work(s) they create with the dataset.

OPEN DATA COMMONS OPEN DATABASE LICENSE - ODC-ODBL

This license allows users to use, share, and adapt a dataset but only if they give credit to the dataset and show any changes or transformations they make to the dataset. Users might not want to use this license because they have to share the work they did on the dataset.

CREATIVE COMMONS ATTRIBUTION-NONCOMMERCIAL 4.0 INTERNATIONAL - CC BY-NC

This license is a restrictive license. Users can share and adapt a dataset, provided they give credit to its creator(s) and ensure that the dataset is not used for any commercial purpose.

CREATIVE COMMONS ATTRIBUTION-NO DERIVATIVES 4.0 INTERNATIONAL - CC BY-ND

This license is also a restrictive license. Users can share a dataset if they give credit to its creator(s). This license does not allow additions, transformations, or changes to the dataset.

CREATIVE COMMONS ATTRIBUTION-NONCOMMERCIAL-SHAREALIKE 4.0 INTERNATIONAL - CC BY-NC-SA

This license allows users to share a dataset only if they give credit to its creator(s). Users can share additions, transformations, or changes to the dataset, but they cannot use the dataset for commercial purposes.

CREATIVE COMMONS ATTRIBUTION-NONCOMMERCIAL-NODERIVATIVES 4.0 INTERNATIONAL - CC BY-NC-ND

This license allows users to share a dataset only if they give credit to its creator(s). Users are not allowed to modify the dataset and are not allowed to use it for commercial purposes.

Note: Additional license types exist. Any dataset you use will include details about its license.

Community data license Agreement

Many data sets were shared under open-source software licenses that were not designed to cover specific considerations related to the distribution and use of data sets.

The Linux Foundation created the **Community Data License Agreement, or CDLA**. <https://cdla.dev/>

CDLA-Sharing

Grants you permission to **use and modify** the data. If **you publish** your modified version of the data, you must do so **under the same license terms as the original data**.

CDLA-Permissive

Grants you permission to **use and modify** the data, You are not required to share changes to the data.

Note that neither license imposes **any restrictions on results you might derive** by using the data, which is important in data science. Let's say, for example, that you are building a model that performs a prediction. If you are training the model using CDLA-licensed data sets, **you are under no obligation to share the model** or to share it under a specific license if you choose to share it.

Summary

Open data is fundamental to Data Science.

Community Data License Agreement makes it easier to share open data.

Open datasets might not meet enterprise requirements, due to the impact they might have on the business.

Sharing Enterprise data – Data Asset eXchange (DAX).

Now there are many open data sets available to the public, but it can be **difficult to find datasets** that are both **high quality** and have clearly defined license and usage terms.

IBM created **DAX**, data asset exchange, **provides a curated collection of open data sets** (images, video, text and audio), under a CDLA, both from IBM research and trusted third party sources, ready for use in enterprise applications.

DAX makes it easier for developers to get started with data sets, provides **tutorial notebooks** that walk through the **basics** of data cleaning, pre processing and exploratory analysis or **more complex tasks** like creating charts, training machine learning models, integrating deep learning via the model asset exchange, and running statistical analysis and time-series analysis.

The Data Asset Exchange and the Model Asset Exchange are both available on the IBM Developer Website.

With these resources, developers can create end to end analytic and machine learning workflows and consume open data and models with confidence under clearly defined license terms.

In the Data Asset Exchange, multiple open data sets are available for you to explore.

Let's say you found a data set that might be very interesting to you, the NOAA weather data, JFK airport database, which contains data from a weather station at the John F. Kennedy Airport in New York. On this data set page, you can click get this data set to **download the NOAA data set** from the cloud storage, **run data set notebooks** to access the notebooks associated with the data set in Watson and **preview the data** and notebooks to explore DAX metadata, glossary and the notebook.

Now, most data sets on DAX are complemented by one or more notebooks.

Click **assets to view all the Jupyter notebooks** and data available. You can click the source code to view all the notebooks associated with your NOAA project. You can execute all the notebooks in Watson Studio to perform data cleaning, pre processing and exploratory analysis.

Data sets on DAX also consist of one or more data files.

Click the data option to view the data files available in your project.

Machine Learning models – Learning from models to make predictions.

What is a machine learning model?

Data contains a wealth of information to solve certain types of problems.

ML uses **algorithms** – also known as “models” - **to identify patterns in the data, that is “model training”**.

A **model trained is used to make predictions** or decisions based patterns learned from past data.

Supervised Learning.

A human provides input data and correct outputs.

The **model** tries to **identify** relationships and **dependencies** between the **input** data and the **correct output**.

Regression models

Used to **predict a numeric** (or “real”) value.

Classification models

Used to predict whether some information or data **belongs to a category** (or “class”).

Unsupervised Learning

Data is **not labeled** by a human. Models identify patterns and structure within the data **based on its characteristics**. Divide each record of a dataset into one of a similar group

Clustering models

Purchase recommendation, anomaly detection.

Reinforcement Learning.

model learns **the best set of actions** to take, given its current environment, to get **the most rewards** over time. Very successful in beating the best human players in games such as Go, chess and popular strategy video games.

Deep Learning

Models and techniques that loosely **emulate the way the human brain** solves a wide range of problems (analyze natural language (both spoken and text), images, audio, video, to forecast time series data).

It **requires large datasets of labeled data** to train a model, is **compute intensive**, and usually requires **special purpose hardware** to achieve acceptable training times.

You can build a custom Deep Learning model from **scratch** or use **pre-trained** models from public model repositories.

Deep Learning models are implemented using popular **learning frameworks** (TensorFlow, PyTorch and Keras) that provide a Python, C++ and JavaScript API.

You can **download pre-trained state-of-the-art models** from repositories (published by academic and commercial research groups) that are commonly referred to as **model zoos**. (TensorFlow, PyTorch, Keras, and ONNX (**Open Neural Network Exchange**)).

Using models to solve a problem

Briefly **outline the high-level tasks** to identify objects in images by **training a deep learning model**.



Collect and label the raw training data that will be used to train a model. Data preparation can be a **time-consuming and labor-intensive process**.

Build a model from **scratch** or select an **existing model** that might be well suited for the task from a public or private resource.

Train the model on your prepared data how to identify objects that are depicted in an image.

Analyze the training results and **repeat the process until the trained model performance meets your requirements**.

When the trained model performs as desired, you **deploy it** to make it available to your applications.

Summary

Machine learning (ML) uses algorithms – also known as “models” – to identify patterns in the data.

The process by which the model learns data patterns is called “model training”.

Types of ML are Supervised, Unsupervised, and Reinforcement.

Supervised learning comprises two types of models, regression and classification.

And deep learning refers to a general set of models and techniques that loosely emulate the way the human brain solves a wide range of problems.

The Model Asset eXchange (MAX).

The Model Asset eXchange or MAX on the IBM Developer platform is a **free open-source** repository for **pre-trained ready-to-use** and custom-trainable deep-learning **microservices**.

The tasks needed to train a model from scratch require a large amount of data, labor, time, and resources and because of this, **time to value can be quite long**.

To reduce time to value, consider **taking advantage of pre-trained models** for certain types of problems.

All models in MAX are **available under permissive open-source licenses**, making it easier to use them for personal and commercial purposes, which **reduces the risk of legal liabilities**.

On MAX, you can find models for a variety of domains, including object detection, image, audio, video, and **text classification, named entity recognition, image-to-text translation, human pose detection**, and more.

These models have been **fully tested** and can be quickly deployed in local and Cloud environments.

How are models Created?

Models are created by **running data through a model using compute resources and domain expertise**.

After **research, evaluation, test, train, and validate steps are complete**, you will have a validated model.

Typical model-serving microservice.

Each microservice includes a **pre-trained deep learning model**, code that **pre-processes the input** before it is analyzed by the model, code that **post-processes** the model output, and a standardized **public API** that makes the services functionality available to applications.

Model-serving microservices are created by running inputs through a validated model and then applying the output to a REST API.

After **implement, package, document, and test** steps are complete, you will have a model-serving microservice that can be sent to a **local machine** or a private hybrid or public **Cloud**.

MAX model-serving microservices

Built and distributed as open-source **Docker images**. Docker is a container platform that makes it easy to build and deploy applications.

The Docker image source is published on **GitHub** and can be downloaded and customized for use in personal and commercial environments.

Use the **Kubernetes open-source** system to automate the deployment, scaling, and management of these Docker images.

Red Hat OpenShift is a popular **enterprise-grade** Kubernetes platform. It is available on IBM Cloud, Google Cloud Platform, Amazon Web Services, and Microsoft Azure.

Summary

- The Model Asset eXchange is a free open source repository for ready-to-use and customizable deep learning microservices
- To reduce time to value, consider taking advantage of pre-trained models for certain types of problems
- MAX model-serving microservices are built and distributed on GitHub as open source Docker images
- Red Hat OpenShift is a Kubernetes platform used to automate deployment, scaling, and management of microservices

Exercise 1

Open the NOAA Weather Data dataset

(<https://developer.ibm.com/exchanges/data/all/jfk-weather-data/>),

Exercise 2

The Model Asset Exchange is a curated repository of open source deep learning models for a variety of domains, such as text, image, audio, and video processing.

For more details, please visit - <https://github.com/CODAIT/max-central-repo> webpage.

The curated list includes deployable models, which you can run as a microservice locally or in the cloud on Docker or Kubernetes, and trainable models where you can use your own data to train the models. Some of the models are already built for you to test. Let's test one of the models.

In this exercise, we explore the Object Detector model hosted on CodePen platform. This model recognizes the objects present in an image. The model consists of a deep convolutional net base model for image feature extraction, together with additional convolutional layers specialized for the task of object detection, trained on the COCO data set. The input to the model is an image, and the output are extracted objects from the image, appropriately labeled.

CodePen is a social development environment. At its heart, it allows you to write code in the browser and see the results of it as you build. It is a useful and liberating online code editor for developers of any skill and is particularly empowering for people learning to code.

1. Navigate to [CodePen](#) webpage.
2. Select **MAX TFJS models** as shown in the screenshot below. Here we are using **Image Segmenter**, which divides an image into regions or categories

that correspond to different objects or parts of objects. Every pixel in an image is allocated to one of a number of these categories.

Summary

- Python offers a diverse library ecosystem for data science, covering scientific computing (Pandas, NumPy), visualization (Matplotlib, Seaborn), and high-level machine learning (Scikit-learn). These libraries offer tools for data manipulation, mathematical operations, and simplified machine learning model development.
- Application Programming Interfaces (APIs) facilitate communication between software components. REST APIs, specifically, facilitate internet communication and access resources like storage. Key API terms include client (user or code accessing it), resource (service or data), and endpoint (API's URL).
- Machine learning models analyze data and identify patterns to make predictions and automate complex tasks—the three fundamental types of machine learning are supervised, unsupervised, and reinforcement learning. Supervised learning includes regression and classification models for predictive modeling and pattern recognition. Deep learning, an advanced subset of machine learning, mimics the brain's processing, enabling intricate problem-solving in various domains.
- The Community Data License Agreement (CDLA) facilitates open data sharing by providing clear licensing terms for distribution and use, and the IBM Data Asset eXchange (DAX) site contains high-quality open data sets.
- The Model Asset eXchange (MAX) provides a wealth of pre-trained deep learning models, empowering developers with readily deployable solutions for various business challenges.

MODULE 4: Jupyter Notebooks and JupyterLab

This module introduces the Jupyter Notebook and JupyterLab. You will learn how to work with **different kernels and the basic Jupyter architecture**. In addition, you will identify the tools in an Anaconda Jupyter environment. Finally, the module overviews cloud-based Jupyter environments and their data science features.

Learning Objectives

- Describe how to use the notebooks in JupyterLab.
- Describe how to work in a notebook session.
- Describe the basic Jupyter architecture.

- Describe how to work with kernels.
- Identify tools in Anaconda Jupyter environments.
- Describe cloud-based Jupyter environments and their data science features.

Introduction to Jupyter notebooks.

Origin of Jupyter

Originated as “iPython” for Python programming, was renamed Jupyter when it started supporting Julia, Python, and R.

What is a Jupyter Notebook?

Is a **browser-based application** to create and share documents containing **code, equations, visualizations, narrative text links, and more.**

Records all steps for data experiments and results that others can reuse.

you can then export the notebook to a **PDF or HTML** file format that can then be shared with anyone.

JupyterLab.

Is an open source **browser-based application** that allows you to access **multiple Jupyter Notebook files**, other code, and data files, compatible with several file formats like CSV, JSON, PDF, Vega, and so on.

Using JupyterLab



Cloud-based services like IBM and Google Colab. **don't require any installation on your local machine.**

They give you access to the Jupyter Notebook environment and allow you to import and export notebooks using the standard **IPython Notebook file format.**

Also, these services **support** the Python language and **other languages as well.**

Jupyter Notebooks can be installed via the command line using the pip **install** function.

It can also be **downloaded locally** on your laptop through the Anaconda Platform from Anaconda dot com. Anaconda is one of the popular distributions which includes Jupyter and Jupyterlab.

So, **for this course**, you have access to a hosted version of JupyterLab in **Skills Network Labs**, so you do not require any installations on your own device to complete the hands-on labs.

Summary

In this video, you learned that:

- Jupyter Notebooks are used in Data Science for recording experiments and projects
- Jupyter Lab is compatible with many files and Data Science languages
- There are different ways to install and use Jupyter Notebooks

Getting started with Jupyter.

Presenting your notebook

Skills Network Labs (SN Labs) is virtual environment. A third-party tool, Lab – Jupyter Notebook configures to use my username and email as credentials..

Click the Open tool tab.

change the name (file/rename notebook).

Run menu

- Run button, runs one cell
- Run selected Cells (Shift + Enter)
- Run all cells

Insert new cell (plus symbol in the tool bar).

Delete a cell.(Edit/delete cells or DD)

Move the cells up or down.

Multiple notebooks(plus button on the toolbar)

Place the notebooks **side by side**.

Create a Markdown(click Code and select Markdown).

You can create line plots and convert each cell and output into a slide or sub-slide in the form of a presentation.

The **slides functionality** in Jupyter allows you to deliver code, visualization, text, and outputs of the executed code as part of a project.

Shut them down. second icon from the top on the sidebar

Shutting down notebooks release their memory, you will see “no kernel” at the top right.

Jupyter kernels.

Computational engine that executes a Notebook file that can be coded in many [languages](#).

Opening a Notebook **launches the related kernel**.

The **top right corner** of the Notebook shows the name of the kernel.

The Skills Network virtual Jupyter environment has Apache, Julia, R, and Swift.

You can use any language to execute your code, either by selecting the kernel on the launch page or clicking the top right icon and selecting the kernel from the drop-down menu.

If running the **kernel on your local machine, you will need to manually install the languages** through your command line interface (CLI).

Markdown

Italics *Italics* or _Italics_

Bold **Italics** or __Italics__

Bolditalic ***Italics*** or ___Italics___

Link [Name of the link](link url)

Image ![alt text](path image)

Table Hyphens(-) for column header, pipe (|) to separate column , new line to separate rows

Unordered list dashes (-), asterisks(*) or Plus(+)

Ordered list numbers followed by periods

Jupyter Architecture.

Jupyter architecture is a **two-process model** with a kernel and a client.

The **client** is the user interface (browser) to send code to the kernel.

The **kernel** executes the code and returns the result to the client.

Jupyter Notebooks represent your code, metadata, contents, and outputs.

When you save the Notebook, it is sent from your browser to the Notebook server. It saves the notebook file on a disk as a

The **Notebook server** is responsible for saving (JSON file with a .ipynb (pronounced as dot i PI NB) extension) and loading the notebooks.

nbconvert tool to convert files to other formats. (preprocessor, Conversor, postprocessor)

Working with Files

Run selected cell => shift + enter

Run selected cell and insert below => alt + enter

Open from URL

Renews notebook

Download Notebook

Running Terminals and Kernels

Shut down all

New View for Notebook

Additional Anaconda Jupyter environmets.

Computational notebooks combine code, computational output, explanatory text, and multimedia resources into a single document.

Jupyter Notebook is computational notebook dozens of programming languages.

Enviroments on local device

Anaconda is a free to install and **open-source distributor** for Python and R, +1500 libraries (installable withous CLI) with free community support.

JupyterLab

JupyterLab is an **open-source, web-based application on the local host** based on Jupyter Notebook. JupyterLab includes expanded features with some of Anaconda's most extensive pre-installed Python libraries, including NumPy, Pandas, and Matplotlib.

URL, which shows the filename and the kernel. It also shows the **Last Checkpoint**.

VS Code

VS Code is a free, **open-source code editor** multiple languages, syntax highlighting, auto-indentation, and more.

If you prefer to install VS code separately, without using Anaconda Navigator, it will not configure for Anaconda, Python, or Jupyter Notebooks.

- Jupyter is a popular computational notebook tool because it supports dozens of programming languages
- The Anaconda Navigator GUI can launch multiple applications on a local device
- Jupyter environments in the Anaconda Navigator include JupyterLab and VS Code
- You can download Jupyter environments separately from the Anaconda Navigator, but they may not be configured properly

Additional Cloud Based Jupyter Environments.

JupyterLite.

Lightweight tool from JupyterLab components **executes entirely in the browser**.

It is a distribution of JupyterLab. <https://jupyter.org/try-jupyter/lab/> in the URL field.

We know this is a JupyterLite notebook because we see the **kernel is Python Pyodide**. (Python distribution for the browser and Node.js based on WebAssembly/[Emscripten](https://emscripten.org/).)

Pyodide is better suited for web applications and data science tasks in the browser, while Pyolite is better suited for desktop applications that need to be distributed as standalone executables

The default kernel for JupyterLite is Pyolite. Pyolite is a Python kernel based on Pyodide. Pyolite runs in the background, so that intensive computations can execute quickly.

GoogleColaboratory.

A free Jupyter notebook environment that runs entirely in the cloud execute on a browser, and GoogleColab projects are stored on Google Drive and GitHub.

Jupyter Notebooks on the internet

There are thousands of interesting <https://github.com/jupyter/jupyter/wiki>

First, you start with exploratory data analysis, for which this notebook is highly recommended: https://nbviewer.jupyter.org/github/Tanu-N-Prabhu/Python/blob/master/Exploratory_data_Analysis.ipynb

For data integration/cleansing at a smaller scale, the python library `pandas` is often used. For this task, you can have a look at this notebook:

<https://towardsdatascience.com/data-cleaning-with-python-using-pandas-library-c6f4a68ea8eb>

If you want to know more about clustering, have a look at this notebook:

https://nbviewer.jupyter.org/github/temporaer/tutorial_ml_gkbionics/blob/master/2%20-%20KMeans.ipynb

And finally, if you want an in-depth notebook on the `_iris_dataset`, have a look at this: <https://www.kaggle.com/lalitharajesh/iris-dataset-exploratory-data-analysis>

MODULE 5: RStudio & GitHub

This module will start with an introduction to R and RStudio and will end up with Github usage. You will learn about the different R visualization packages and how to create visual charts using the plot function.

Further in the module, you will develop the essential conceptual and hands-on skills to work with Git and GitHub. You will start with an overview of Git and GitHub, creating a GitHub account and a project repository, adding files, and committing your changes using the web interface. Next, you will become familiar with Git workflows involving branches, pull requests (PRs), and merges. You will also complete a project at the end to apply and demonstrate your newly acquired skills.

Learning Objectives

- Describe R capabilities and RStudio environment.
- Use the inbuilt R plot function.
- Explain version control and describe the Git and GitHub environment.
- Describe the purpose of source repositories and explain how GitHub satisfies the needs of a source repository.
- Create a GitHub account and a project repository.
- Demonstrate how to edit and upload files in GitHub.
- Explain the purpose of branches and how to merge changes.

Introduction to R and RStudio

R is an easy to use **statistical programming language** for data processing and manipulation, statistical inference, data analysis, and machine learning algorithm, producing great visualizations.

Import data from different sources like flat files, databases, web, and statistical software such as SPSS and STATA.

RStudio. Is an **IDE** for R with a syntax-highlighting editor that supports direct code execution and a place where you can keep a record of your work, a **Console** for typing R commands, a workspace and **History** tab that shows the list of R objects you created during your R session and the history of all previous commands, and finally, Files, Plots, Packages, and Help tabs.

The Files tab shows files in your working directory.

The Plots tab displays the history of plots you have created.

You can also export plots to PDF or image files.

The Packages tab displays external R packages available on your local computer.

R libraries available:

- dplyr for manipulating data,
- stringr for manipulating strings,
- ggplot for visualizing data, and
- caret for machine learning.

shortcuts

Clear console > ctrl +L

Run source > ctrl + shift + S

Run selected > Ctrl + Enter

Plotting in RStudio

Welcome to “Plotting in RStudio.”

After watching this video, you will be able to:

List the R data visualization packages,

Use the inbuilt R plot function,

Use the R ggplot library to add functions and arguments to the plot,

And, add titles and names to the plot.

With the influx of data, one of your many jobs as data scientists is to produce insights using visualizations.

R has different packages for data visualization that you can use based on your requirement.

To install these packages:

```
>install.packages <package name>
```

Ggplot: used for data visualizations such as histograms, bar charts, scatterplots, and so on. It allows adding layers and components to a single visualization.

Plotly: is used for web-based data visualizations that can be displayed or saved as individual HTML files.

Lattice: is used to implement complex, multi-variable data sets. It is a high-level data visualization library that can handle graphics without customizations.

Leaflet: is used for creating interactive plots.

Inbuilt functions

Plot() function returns a scatterplot of the values vs. the index.

Title() to make the visualization easier to read and understand.

Library() read the argument library into the memory.

ggplot library of R.

Adds layers to plots using different functions and arguments.

```
library(ggplot2)
ggplot(mtcars, aes(x=mpg, y =
wt))+geom_point()
```

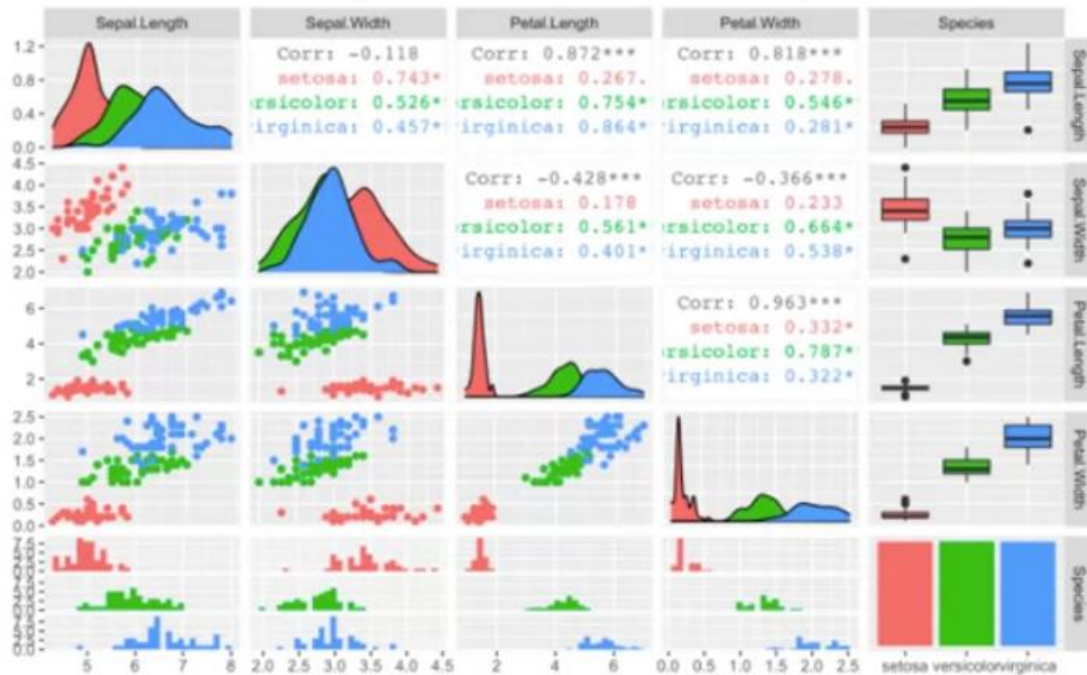
For example, to create a scatter plot, let's use the inbuilt dataset Mtcars.

using the library function. Next, use the ggplot function on the data frame MTcars, specify the **X axis** as miles per gallon and the **Y axis** as weight. Then add the **geom point function to specify a scatter plot**; otherwise, it will return an empty plot.

```
ggplot(mtcars, aes(x=mpg, y = wt))+geom_point() + ggtitle("Miles per
gallon vs weight") + labs(y="weight", x = "Miles per gallon")
```

Add titles and change the axis name by using the **ggtitle()** argument and the **labs()** argument to specify appropriate names for both axes.

In the lab, you will recreate the graphics with ggplot and the extension library called GGally.



GGally extends ggplot by adding several functions to reduce the complexity of combining geometric objects with transformed data.

LAB 1

```
//Select library
```

```
library(datasets)
```

```
// Load dataset mtcars
```

```
data(mtcars)
```

```
// show 10 first lines from dataset
```

```
Head(mtcars, 10)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4

```
// show metadata
```

```
?mtcars
```

Motor Trend Car Road Tests

Description

The data was extracted from the 1974 *Motor Trend* US magazine, and comprises fuel consumption and 10 aspects of automobile design and performance for 32 automobiles (1973–74 models).

Usage

`mtcars`

Format

A data frame with 32 observations on 11 (numeric) variables.

[, 1] mpg Miles/(US) gallon
[, 2] cyl Number of cylinders
[, 3] disp Displacement (cu.in.)
[, 4] hp Gross horsepower
[, 5] drat Rear axle ratio
[, 6] wt Weight (1000 lbs)
[, 7] qsec 1/4 mile time
[, 8] vs Engine (0 = V-shaped, 1 = straight)
[, 9] am Transmission (0 = automatic, 1 = manual)
[,10] gear Number of forward gears
[,11] carb Number of carburetors

Note

Henderson and Velleman (1981) comment in a footnote to Table 1: 'Hocking [original transcriber]'s noncrucial coding of the Mazda's rotary engine as a straight six-cylinder engine and the Porsche's flat engine as a V engine, as well as the inclusion of the diesel Mercedes 240D, have been retained to enable direct comparisons to be made with previous analyses.'

Source

Henderson and Velleman (1981), Building multiple regression models interactively. *Biometrics*, **37**, 391–411.

```
// load ggplot package
```

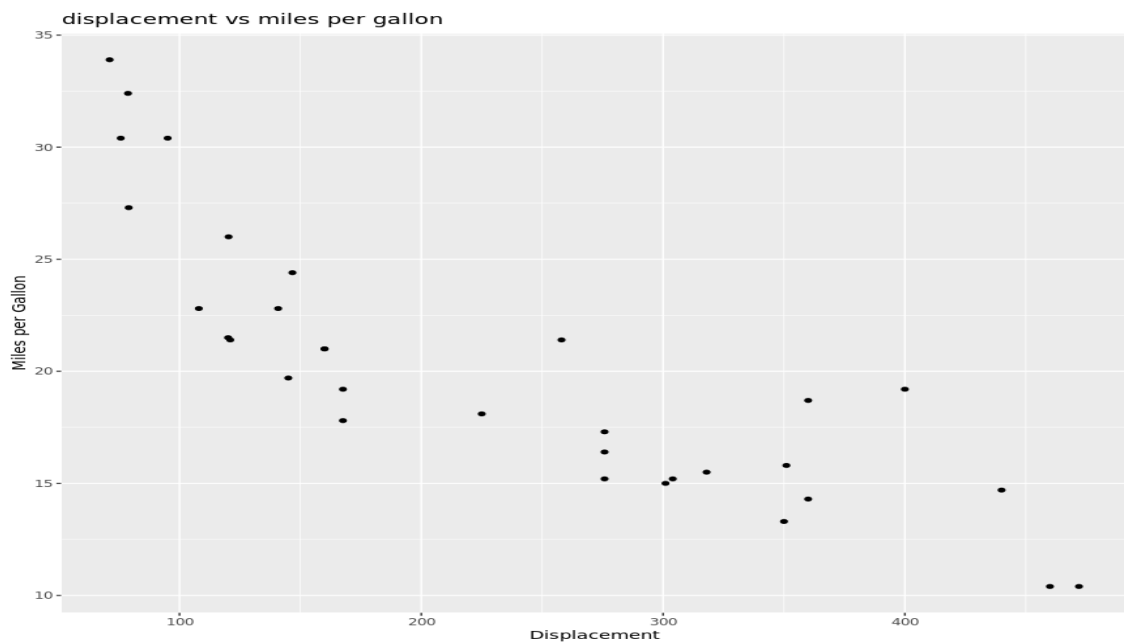
```
library(ggplot2)
```

```
//create scatterplot between disp on x-axis and mpg on y-axis  
ggplot(aes(x=disp,y=mpg,),data=mtcars) + geom_point()
```

```
//add a title +ggtitle("displacement vs miles per gallon")
```

```
// add axis labels +labs(x = "Displacement", y = "Miles per Gallon")
```

```
ggplot(aes(x=disp,y=mpg,),data=mtcars)+geom_point()+ggtitle("displacement vs  
miles per gallon") + labs(x = "Displacement", y = "Miles per Gallon")
```



```
// convert variable vs, into a categorical variable having two levels.
```

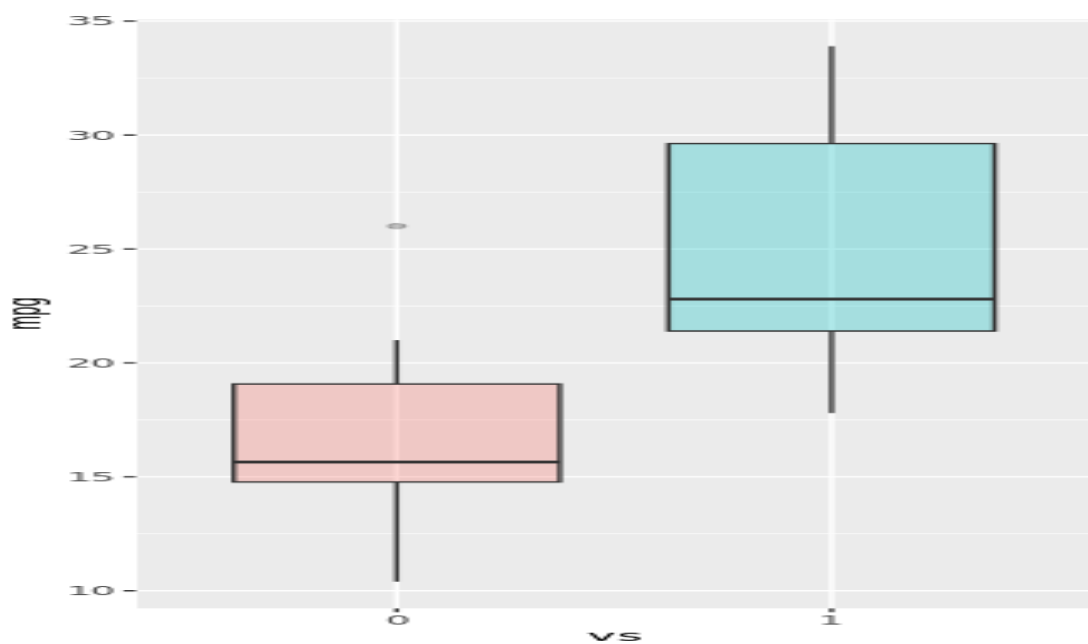
```
mtcars$vs <- as.factor(mtcars$vs)
```

```
// create a box plot per each category in vs, showing mpg distribution, filling each box with a different color according to the category vs and an alpha channel (transparency of 0.3) . No legend shown.
```

```
ggplot(aes(x=vs, y=mpg, fill = vs), data = mtcars) +
```

```
  geom_boxplot(alpha=0.3) +
```

```
  theme(legend.position="none")
```



//plot an **histogram** for weight variable having bin width of 500 pounds

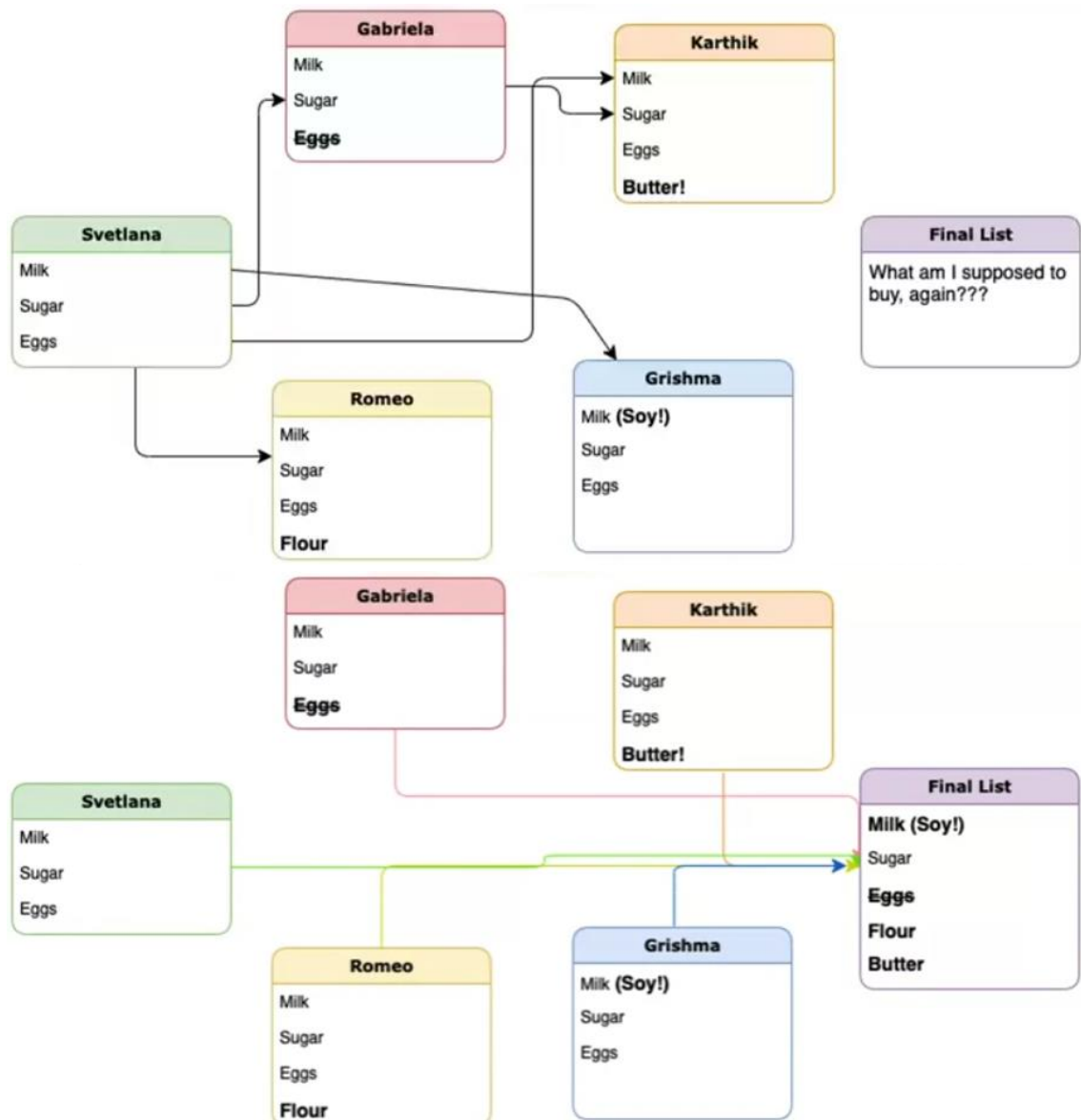
```
ggplot(aes(x=wt),data=mtcars) + geom_histogram(binwidth=0.5)
```



Overview of Git/Github

Git and GitHub perform **version control** of any number of file types (of source code files, images, documents) and projects and collaborating with others.

A version control system allows you to **keep track of changes** to documents, recover old versions, and collaborate easily with others.



Git is a **distributed** version control system free and open source software distributed under the GNU General Public License.

You can use Git using your command line interface, but there are several web interfaces: GitHub, GitLab, BitBucket, and Beanstalk.



Short Glossary Terms

SSH protocol: secures remote login from one computer to another.

Repository: contains your project folders that are set up for version control.

Fork: is a copy of a repository.

Pull request: is the way you **request** that someone reviews and **approves** your **changes** before they become final.

Working directory contains the files and subdirectories on your computer that are associated with a Git repository.

Basi Git Commands

"git clone" Copy an existin repository

"git init". Creates one empty repository locally.

"git add" moves changes from the working directory to the staging area.

"git status" allows you to see the state of your working directory and the staged snapshot of your changes.

"git commit" takes your staged snapshot of changes and commits them to the project.

"git reset" undoes changes that you've made to the files in your working directory.

"git log" enables you to browse previous changes to a project.

"git branch" lets you create an isolated environment within your repository to make changes.

"git checkout" lets you see and change existing branches.

"git merge" lets you put everything back together again.

Learnnig Resources

[TCheat Sheet](#)

[Visualizing git](#)

[Handbook](#)

[Handbook](#)

Introduction to GitHub

Background of git

Linux development in the early 2000's was managed under a free-to-use system known as BitKeeper. In 2005, **BitKeeper changed to a for-fee** system which was problematic for Linux developers for many reasons. Linus Torvalds led a team to develop a **replacement source-version control** system.

The project ran in a short timeframe and the key characteristics were defined by a small group:

Strong support for non-linear development. ((Linux patches were then arriving at a rate of 6.7 patches per second)

Distributed development. (Each developer can have a local copy of the full development history.

Compatibility with existing systems and protocols. (This was necessary to acknowledge the diversity of the Linux community.

Efficient handling of large projects.

Cryptographic authentication of history. (This makes certain that distributed systems all have identical code updates.

Pluggable merge strategies. (Many pathways of development can lead to complex integration decisions that might require explicit integration strategies.)

Git Repository Model

Git is designed as a **distributed version-control system**, primarily focused on **tracking source code during development**.

Contains elements to **coordinate among programmers**, track changes, and support non-linear workflows.

Created in 2005 by Linus Torvalds for distribution of Linux kernels.

What is Git?

Git is a **distributed version-control system** that is used to **track changes** to content. It serves as a **central point for collaboration** with a particular focus on **agile development methodologies**.

In a central version control system, every developer needs to check out code from the central system and commit back into it.

As Git is a distributed version control, **each developer has a local copy of the full development history**, and changes are copied from one such repository to another. **Each developer can act as a hub.**

When Git is used correctly, there is a **main branch that corresponds to the deployable code.**

Teams can continuously integrate changes that are ready to be released and can simultaneously **work on separate branches in between releases.**

Git also allows **centralized** administration of tasks with **access-level controls for each team.**

Git can co-exist locally such as through the **GitHub Desktop client** or it can be used directly through a browser connected to the GitHub web interface.

IBM Cloud is based on sound and established open-source tools including Git repositories, often called repos.

What is GitHub?

GitHub is an **online hosting service for Git repositories.**

GitHub hosted by a subsidiary of Microsoft.

GitHub offers **free, professional and enterprise accounts.**

As of August 2019, GitHub had over 100M repositories.

A Repository is a **data structure for storing documents** including application source code. A repository can track and maintain version-control.

What is GitLab?

GitLab is a **complete DevOps platform, delivered as a single application.**

GitLab provides access to Git repositories, controlled by source code management.

With GitLab, developers can:

Collaborate, reviewing code, making comments and helping to improve each other's code. Work from their own local copy of the code. Branch and merge code when required. **Streamline testing and delivery with Built-in Continuous Integration (CI) and Continuous Delivery (CD).**

GitLab Repositories

Signing up for a free, personal account on GitHub at the GitHub site, <https://github.com>. Join a free plan and select the type of account – most likely a free, personal account is all that you'll want.

Choose to set up a personal, free account, which is the default.

Respond to an email that you receive which proves that you linked to GitHub from an account that you access

GitHub provides you with some starting points.

You can choose to create a repository or an organization, or you can take the Introduction to GitHub course.

Remember, a **repository is a data structure for storing documents** including application source code which tracks and maintains version-control.

An organization is a collection of user accounts that owns repositories. Organizations have **one or more owners**, who have administrative privileges for the organization.

GitHub provides many resources to help you work effectively. When you have time, read the GitHub guide.

The **heart of a Git-based project is the repository**. This contains all your code and the related artifacts, including things like: A **README** file to describe the purpose of the project. A **license** to express the ways in which people can use your code, Etc. You can also make your repository private (**only available to people with accounts that have permission to see it**) or public (searchable and seen by everyone).

A repository has **a number of tabs**, and is opened to the Code tab.

Code – this is where all the source files reside. Git was initially created as a source code repository and now all sorts of files end up in here. If you created a README and/or license, that's all that's here right now.

Issues – as you can imagine, you can track and plan with tools such as “Issues” that lists all open items against your project base.

Pull Requests – this is part of the mechanism for collaborating with other users. Pull requests define changes that are committed and ready for review before being merged into the main branch.

Projects – all the tools for managing, sorting, planning, etc. your various projects. This is the core of the collaborative power of GitHub.

Wiki, Security, and Insights – often left for more advanced users, these tools provide a communication base to the external user community.

Settings – GitHub allows for a lot of personalization, including changing the name of your repository and controlling access.

GitHub Getting Started

Creating a repository

For creating a new repository provide:

Name. Description. Visibility (public or private), Initialize a readme file.

The **root folder of your repository is listed** by default and it has just one file ReadMe.md.

Now, it's time to **edit the readme**.

Commit them. After you have made your changes, scroll down to the Commit changes section. Add a **commit message** and optionally add a **description**, then click Commit changes. The

"commit changes" is used to save your changes to the repository.

Creating a new file

Using the built-in web editor provided by GitHub which runs in the browser.

Click Add File, then click Create New File to create the new file.

To create a python file called firstpython.py.

Add a comment, add the code.

Once finished, **commit** the change to the repository. You can see that your file is now added to the repository and the repository listing shows when the file was added or changed.

When you need to change the file, you can edit it again.

Click the file name, and then click the pencil icon, make your edits and commit the changes.

Upload a file

You can also **upload a file** from your local system into the repository.

Once the files finish uploading, click **Commit Changes**. The repository now reflects the files that were uploaded.

GitHub working with Branches

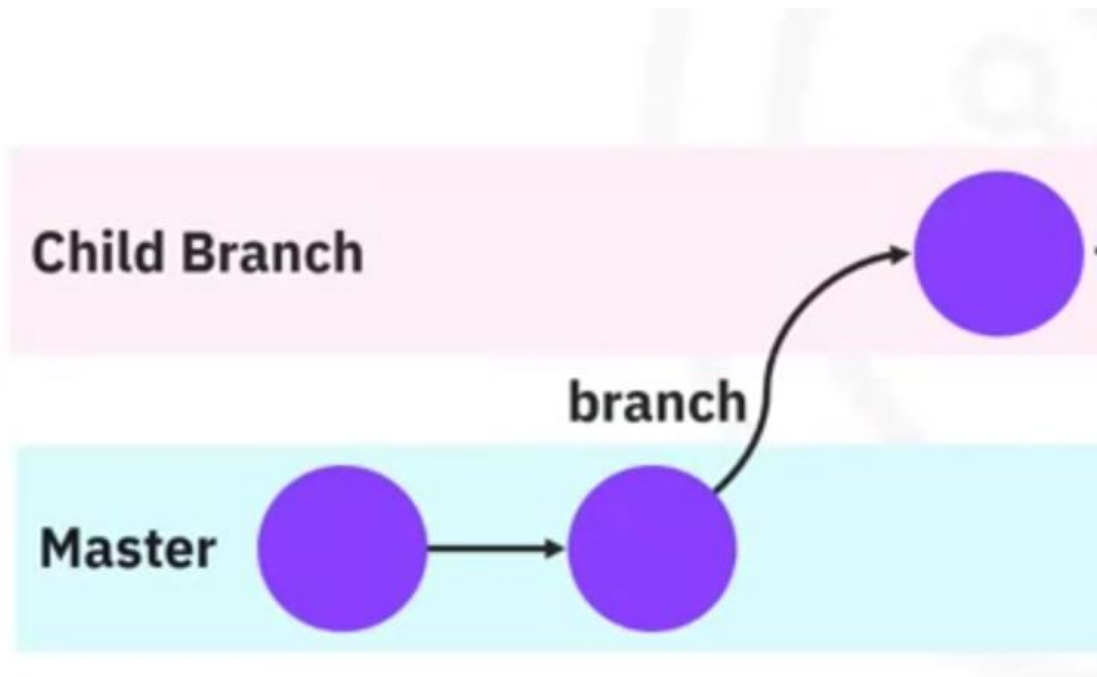
What is a branch?

A branch is **a snapshot of your repository to which you can make changes.**

It **is a copy of the master branch** and can be used to develop and test workflow changes before merging it into the master branch.

The main branch called **master**. is the official working version of your project having It has the deployable code.

It is advisable not **to push any code that has not been tested in the master.**



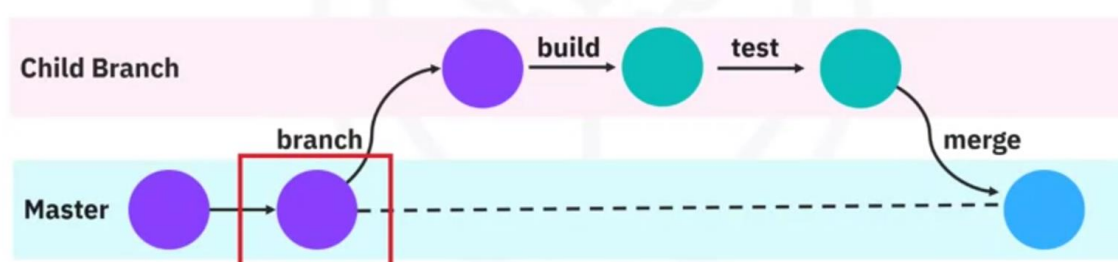
Why create a Branch?

If you want **to change the code** and the workflow in the master branch, you can **create a copy of the master branch** (the child branch).

In the child branch, changes and experiments are done.

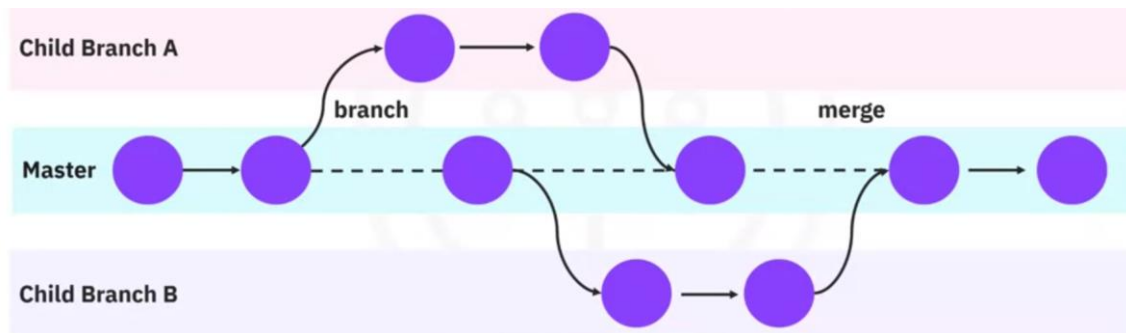
You can build, make edits, test the changes, and when you are satisfied with them, you can **merge them back to the master branch**, where you can prepare the model for deployment.

You can see that all of this is **done outside the main branch** and until you merge, changes will not be made to the workflow before you branched.



Merging multiple branches

To ensure that **changes done by one member, do not impede or affect the workflow of other members**, multiple branches can be created and merged appropriately with the master after the workflow is properly tested and approved.



How to create a branch

Click the drop-down arrow and **create** a new branch. **Name** the new branch 'child branch' and then click enter.

All the content in the master branch is copied to the child branch.

Add files in the child branch without adding any to the master branch.

Commit new file. **Descriptive commit messages are important as they help to keep track of the changes made.**

The file is added to the child branch and is not added to the master branch.

Pull Request

After test and ensure it is working. You can **merge the changes in the child branch to reflect in the master branch** by creating a Pull Request (PR).

A Pull Request **show the differences** (compare & pull Request) in the content from both branches.

A Pull Request notify other team members of the changes and edits to the main branch.

Ideally, **another team member reviews the changes and approves** them to be merged with the Master branch.

Pull requests are a means of collaboration on GitHub.

When you open a pull request, you propose your changes.

You can **assign team members to review and approve your contribution** and merge in the target branch.

You will now **create the pull request**. Add the **title** and an optional **comment**.

Click Create pull request.

If you are **okay** with the changes, click **Merge pull request** and then click Confirm.

You will get a **confirmation that** the pull request has been **successfully merged**.

You can delete the branch if you no longer need to edit or add new information.

Getting Started with Branches using Git Commands

create a new local repository using git init

Inside my folder project a .git directory show all information the git server requires to control the document's versions

```
total 40
drwxr-sr-x 7 theia users 4096 Sep 16 02:14 .
drwxr-sr-x 3 theia users 4096 Sep 16 02:14 ..
-rw-r--r-- 1 theia users  23 Sep 16 02:14 HEAD
drwxr-sr-x 2 theia users 4096 Sep 16 02:14 branches
-rw-r--r-- 1 theia users  92 Sep 16 02:14 config
-rw-r--r-- 1 theia users  73 Sep 16 02:14 description
drwxr-sr-x 2 theia users 4096 Sep 16 02:14 hooks
drwxr-sr-x 2 theia users 4096 Sep 16 02:14 info
drwxr-sr-x 4 theia users 4096 Sep 16 02:14 objects
drwxr-sr-x 4 theia users 4096 Sep 16 02:14 refs
```

create and add a file to the repo using git add

commit changes using git commit

Before commit configure user.mail and user.name

create a branch using git branch

switch to a branch using git checkout

check the status of files changed using git status

review recent commits using git log

revert changes using git revert

get a list of branches and active branch using git branch

merge changes in your active branch into another branch using git merge

MODULE 6: Create and Share your Jupyter Notebook

In this module, you will work on a final project to demonstrate some of the skills learned in the course. You will also be tested on your knowledge of various components and tools in a Data Scientist's toolkit learned in the previous modules.

Learning Objectives

- Create a Jupyter Notebook with markdown and code cells

- List examples of languages, libraries and tools used in Data Science
- Share your Jupyter Notebook publicly on GitHub
- Evaluate notebooks submitted by your peers using the provided rubric
- Demonstrate proficiency in Data Science toolkit knowledge

MODULE 7: IBM Watson Studio

This is as an optional module if you are interested in learning about and working with data science tools from IBM such as Watson Studio.

Learning Objectives

- Find common resources in Watson Studio and IBM Cloud Pak for Data.
- Create an IBM Cloud account, service, and project in Watson Studio.
- Create and share a Jupyter Notebook.
- Use different types of Jupyter Notebook templates and kernels on IBM Watson Studio.
- Describe how to connect a Watson Studio account and publish a notebook in GitHub.

Introduction to Watson Studio

- Explain what Watson Studio is for and who uses it.
- List the components of IBM Cloud Pak for Data,
- Find common resources in Watson Studio and IBM Cloud Pak for Data
- Build models and manage services and integrations.

Who uses Watson Studio?

Watson Studio is a collaborative platform for the data science community to analyze data and construct models. It is used by:

- Data Analysts.
- Data Scientists.
- Data Engineers.
- Developers.
- Data Stewards.

What can you do with Watson Studio?

- **Create projects** to organize data connections, data assets, and Jupyter notebooks.
- **Upload files** to your project.
- Clean and shape the data to **refine** it for analysis.
- Create and share **data visualizations** via dashboards without using any coding.
- Watson Knowledge Catalog provides a secure enterprise catalog management platform to deliver **trusted and meaningful data**.
- Watson Machine Learning offers tools and services to **build, train, and deploy** machine learning models.

Cloud Pak for Data as a service.

Cloud Pak for Data as a Service is a secure, seamless data access and integration platform that enables a single view of the data, no matter how many data sources you are working with.

It includes

- IBM Watson Studio,
- IBM Watson Knowledge Catalog,
- IBM Watson Machine Learning and more.

You can:

- Find **step-by-step tutorials** that show how to integrate data from multiple sources, build, deploy, test, and more.
- Create **collaborative data workspaces called Projects** where your team can perform tasks for data science, data engineering, data curation, or machine learning and AI.
- Read Cloud Pak for Data news and updates.

As you scroll down, you will see your work highlights, recent activities, and shortcuts.

The **Quick start section** has links to get you started, and the navigation menu is on the upper left.

In the **navigation menu**, Projects shows projects and jobs you have created.

In Deployments, you can train, deploy, and manage machine learning models in collaborative deployment spaces.

In Services, you can view different services associated with your account, and explore the catalog.

and Gallery shows a collection of data sets, notebooks, industry accelerators, and sample projects.

Cloud Pak for Data Gallery

Now, on the Gallery page, you can search for projects and filter your search by clicking All filters

You can filter by format and topics.

You can then explore different project types.

and you can also explore the data.

Once you select the project, you will see the notebook, when it was last modified, the problem statement, and more.

You can Add to your project or download the notebook to your system.

Watson Studio Architecture

One of the core services in Cloud Pak for Data, Watson Studio architecture is centered around the project.

To **create a project**, select **Work with data** from the Cloud Pak for Data homepage.

The Create a project popup will appear with options to create an **empty project** or create a project **from a sample** or file.

Now when you click, Create an empty project, this page loads. Here you can manage all your projects. Use the context information and actions menu from anywhere in the project to view project information or load data files as assets.

Watson Studio IDE Editor

The RStudio integrated development environment (IDE) is included with IBM Watson Studio so you can use R notebooks and scripts in your projects.

Launch RStudio from the Launch IDE menu after you create a project.

The Overview page keeps you up to date with recent assets you created, the resource usage of the project, a readme for your project description, and a project history.

To **find project assets**, click the Assets tab.

The New Assets button lets you use data and tools to create analytical assets, like flows, visualizations, experiments, or notebooks, and the Import Asset button lets you import assets.

Watson Studio jobs tab

A job is a way of running assets, such as Data Refinery flows or Notebooks in a project.

Under the Jobs tab, you can **run** a job immediately or **schedule** a job.

Watson Studio manage Tab

You can manage your projects under the Manage tab.

There you can control access with user groups, define environments and monitor active runtimes, see resource usage, and tools and processing power using Services and integrations.

In Service & Integration, you can **associate IBM Cloud services** with your project to add tools, environments, and capabilities.

And you can also **use third-party Integrations** so your project can interact with external tools.

For example, you can integrate with a Git repository to export the project, work with documents and notebooks in JupyterLab, or back up the project.

Watson Studio new asset tool types.

From the Asset tab, you can use graphical builders, like:

Dashboards Editor, to create a sharable visualization in the dashboard, or **Data Refinery** to create flows to refine data. **Decision Optimization** has the Decision Optimization model builder to solve scenarios, and with **SPSS Modeler**, you can quickly develop predictive models and deploy them into business operations to improve decision-making.

The New asset tool type options also include **Code editors**, which provides a Jupyter notebook editor where you can analyze data or build models. A Jupyter notebook is a web-based environment for interactive computing. You can use notebooks to run small pieces of code that process your data, and you can immediately view the results of your computation. Now, this Jupyter notebook editor is largely used for interactive, exploratory data analysis programming and data visualization. You should use it if you are new to Jupyter notebooks.

Recap

In this video, you learned that:

- Watson Studio is a helpful tool for
 - Analyzing and viewing data
 - Cleaning and shaping data
 - Embedding data into streams
 - Creating, training, and deploying
- Learning Watson Studio promotes career growth
- Learning Watson Studio is easy and requires no special skills
- Watson Studio offers many available resources

My trial

2c5f31f7cc1488abae52a7e85b517186

Glosary

Term Definition

Apache MLib Language that makes machine learning scalable

Apache Spark A general-purpose cluster-computing framework allowing you to process data using compute clusters

API Application programming interface allows communication between two pieces of software

Caffe deep learning algorithm repository built with C++ with Python and Matlab bindings

CDLA Community Data License Agreement

Classification models Are used to predict whether some information or data belongs to a category (or “class”)

CLI Command line interface

C++ A general-purpose programming language. It is an extension of the C programming language or C with Classes

Data set A structured collection of data

Deeplearning4 Language for deep learning

Deep learning A specialized type of machine learning. It refers to a general set of models and techniques that loosely emulate the way the human brain solves a wide range of

problems

ELT Extract, Load, Transform

ETL Extract, Transform, and Load

FSF Free Software Foundation

ggplot2 A popular library for data visualization in R

GPU Graphics processing units

Git De facto standard for code asset management, also known as version management or version control. Around Git emerged several services, GitHub, and GitLab

Hadoop Application of Java which manages data processing and storage for big data applications running in clustered systems

Java Object-oriented programming language

Java-ML Language for machine learning

JVM Java Virtual Machine

JavaScript A general-purpose language that extended beyond the browser with the creation of Node.js and other server-side approaches

Julia A language for high-performance numerical analysis and computational science

Jupyter Notebook A browser-based application that allows you to create and share documents containing code, equations, visualizations, narrative text links, and more

Jupyter Lab A browser-based application that allows you to access multiple Jupyter Notebook files, other code, and data files

Kernel An execution environment for the different programming languages

Lattice It is a high-level data visualization library that can handle graphics without customizations

Library A collection of functions and methods that allow you to perform many actions without writing the code

Leaflet Used for creating interactive plots

ML Machine learning uses algorithms – also known as “models” - to identify patterns in the data

Matplotlib package for data visualization

Model training The process by which the model learns patterns from data

MNIST Modified National Institute of Standards and Technology

MongoDB A NoSQL database for big data management that was built with C++

NLP Natural Language Processing

NLTK Natural Language Toolkit

NumPy Libraries are based on arrays and matrices, allowing you to apply mathematical functions to the arrays

OSI Open-Source Initiative

PaaS Platform as a service

Pandas A library that offers data structures and tools for effective data cleaning, manipulation, and analysis

Plotly Used for web-based data visualizations that can be displayed or saved as individual HTML files

PMML Predictive Model Markup Language

Python A high-level, general-purpose programming language. It has a large, standard library that provides tools suited to many different tasks, including Databases, Automation, Web scraping, Text processing, Image processing, Machine learning, and Data analytics

R A statistical computing language

Regression models Are used to predict a numeric (or “real”) value

Reinforcement

Learning Loosely based on the way human beings and other organisms learn.

REST RE stands for Representational; the S stands for State, and the T stands for Transfer

RStudio Unifies programming, execution, debugging, remote data access, data exploration, and visualization into one tool

SaaS Software as a service

Scala Is a combination of scalable and language. A general-purpose programming language that provides support for functional programming and is a strong static type system

Spyder Integrates code, documentation, and visualizations, among others, into a single canvas

SQL Structured Query Language that is non-procedural, used for querying and managing data

Supervised Learning A learning in which a human provides input data and correct outputs

TensorFlow Deep Learning library for dataflow that was built with C++

Unsupervised Learning The data is not labeled by a human. Examples are Clustering models used to divide each record of a dataset into one of a similar group

Watson Studio A fully integrated development environment for data scientists

Weka Language for data mining